A public services platform serving multiple cities.

Fabio Augusto De Oliveira Dutra — NF1045061

Gumashtha Devangiri Pandukabhaya Punchi Wedikkarage Don — NF3003049

Jose Vidal Asencio Santos — NF1042429

Mehrthur Luís Lemos da Silva — NF3003003

The University of Niagara Falls Canada

Term 1

Term 2

Teacher : Dr. William Pourmajidi

October 13, 2026

Introduction

The Smart Regional Services Platform integrates eight data sources across multiple cities, including operational records, sensor readings, spatial data, multimedia, and historical information. These sources require storage and retrieval strategies suited to different workloads.



Figure 1 - Smart City

This report evaluates database architecture, storage methods, indexing strategies, specialized data models, and contemporary processing paradigms. It recommends PostgreSQL and PostGIS for operational and spatial workloads, MongoDB for environmental data, and S3 for multimedia storage. A separate analytical architecture using S3, Glue, EMR, and Redshift supports regional reporting while protecting operational performance. The recommendations consider scalability, consistency, and maintenance costs.

Part 1 – Platform Requirements Analysis

According to Vazquez and Simões (2016), the main purpose of requirements specification is to document customers' needs accurately and comprehensively and obtain their approval of the proposed product. It also helps the development team understand what customers expect, serving as an important communication tool between customers and developers.

Component	Application to the Platform
Overview	Objective: integrate public service data across multiple cities. Main stakeholders: citizens, municipal teams, emergency response operators, and regional managers.
System Models	System models illustrate the relationships between the platform, citizens, municipal departments, and sensors through a context diagram. The requirements analysis table complements this model by documenting the eight data sources, their data types, expected access patterns, potential performance concerns, and initial database design implications. The architecture diagram is presented in Part 2
Functional Requirements	Register service requests, attach evidence, receive sensor readings, query locations, track maintenance, evaluate emergency rules, and retrieve historical data.
Nonfunctional Requirements	Performance, scalability, availability, security, and data integrity requirements. Numerical targets not provided in the scenario should be identified as assumptions to be validated.
Detailed Requirements Specification	For each function, describe the users, data involved, conditions, rules, and expected outcomes.

The following table identifies the eight data sources, their expected access patterns, possible performance concerns, and initial database design implications.

Data Source / Platform Requirement	Type of Data	Expected Access Pattern	Possible Performance Concern	Initial Database Design Implication
ROAD_AND_FACILITY_LOCATIONS	Spatial points, lines and polygons; structured and variable attributes.	Find nearby facilities, roads intersecting a zone, and locations linked to operational records.	Expensive spatial joins and inconsistent identifiers across stores.	Use PostGIS geometries and spatial indexes; retain stable IDs; justify optional MongoDB attributes against JSONB. See Appendix A, Section A.1, and Appendix B, Figure B1.
MULTIMEDIA	Images and short videos, with structured metadata and contextual links.	Upload once; retrieve metadata and authorized files for a request, inspection or capture.	Large file volumes, upload failures and costly repeated downloads.	Store files in S3 and metadata in PostgreSQL; track upload state, checksum and stable media IDs. See Appendix A, Section A.2, and Appendix B, Figure B2.
TRANSPORTATION_SENSORS	Temporal numerical readings and transit events; device configuration.	Ingest every few seconds; retrieve by sensor, road or intersection, and time range.	Sustained writes, growing history, and slow time-range scans.	Use PostgreSQL operational tables; evaluate time partitioning and sensor/time indexes; keep analytical summaries separate. See Appendix A, Section A.3, and Appendix B, Figure B3.
ENVIRONMENTAL_MONITORING	Temporal numerical measurements, observed events and device documents.	Continuous ingestion; retrieve by device, metric and period; examine threshold events and trends.	High ingestion volume, duplicate deliveries and scans across long periods.	Use MongoDB time-series collections for readings; regular collections for devices/events; validate timestamps, units and references. See Appendix A, Section A.4, and

Data Source / Platform Requirement	Type of Data	Expected Access Pattern	Possible Performance Concern	Initial Database Design Implication
				Appendix B, Figure B4.
CITIZEN_SERVICE_REQUESTS	Structured requests, descriptions, assignments and temporal status history.	Create reports; find open requests by city, status or department; retrieve one request and its history.	Concurrent status changes, growing queues and inefficient filtered lists.	Use PostgreSQL transactions, constrained status transitions and indexes aligned with city/status/department queries. See Appendix A, Section A.5, and Appendix B, Figure B5.
FACILITY_MAINTENANCE	Structured inspections, work orders, teams, costs and equipment usage.	Assign work; list outstanding orders; retrieve facility history and cost totals.	Concurrent updates, multi-table retrieval and expensive historical aggregation.	Use transactional PostgreSQL tables and local FKs; index facility/time and status; move reporting aggregates to analytics. See Appendix A, Section A.6, and Appendix B, Figure B6.
EMERGENCY_RULES_AND_RESPONSE	Versioned rules, conditions, alerts, actions and escalation history.	Retrieve effective rules; evaluate incoming events; find unacknowledged alerts and their history.	Event bursts, repeated alerts and slow rule or notification processing.	Store rules and alerts in PostgreSQL; evaluate through a service with version tracking, deduplication and delivery retries. See Appendix A, Section A.7, and Appendix B, Figure B7.
HISTORICAL_PLANNING	Archived reports, temporal summaries and analytical	Compare cities, facilities and reporting periods; examine seasonal trends and planning indicators.	Large aggregations competing with operational activity; inconsistent or	Use a separate PostgreSQL analytical database with validated incremental loads, shared dimensions and daily summaries.

Data Source / Platform Requirement	Type of Data	Expected Access Pattern	Possible Performance Concern	Initial Database Design Implication
	dimensions and facts.		delayed source data.	See Appendix A, Section A.8, and Appendix B, Figure B8.

Requirements Analysis

This platform is an advanced database problem because its information has different structures, arrival rates, relationships, and uses. A citizen request requires accountable assignment and consistent status updates, while a sensor stream produces repeated observations that are normally retrieved by device and time. A geographic query depends on spatial relationships rather than an identifier alone. Multimedia retrieval also involves transferring files whose size greatly exceeds their metadata. The design must coordinate these workloads while keeping identifiers, timestamps, and access controls consistent across participating cities.

Transportation and environmental readings are likely to create sustained storage growth because devices report repeatedly. Queries over long periods can scan many observations unless records are organized around their access patterns. Incoming messages may arrive late or be delivered again, so ingestion needs validation and duplicate handling. Multimedia evidence presents a different challenge: file size, upload reliability, and repeated downloads. The proposed separation between object storage and a searchable metadata catalogue addresses these differences. Geographic searches and historical aggregations can also become expensive as coverage and reporting periods expand.

Several sources require specialized database thinking. Road geometries, facility positions, and service-zone boundaries need spatial operations supported by PostGIS. Sensor

measurements require temporal organization, appropriate units, and clear treatment of missing observations. Images and videos need an object-storage approach alongside relational metadata. Emergency response depends on versioned conditions and event evaluation; the proposed rule service evaluates those conditions, while PostgreSQL preserves the rules and resulting actions. Flexible environmental documents and optional location attributes may benefit from document storage, although flexibility alone does not justify introducing another database where JSONB would meet the same requirement.

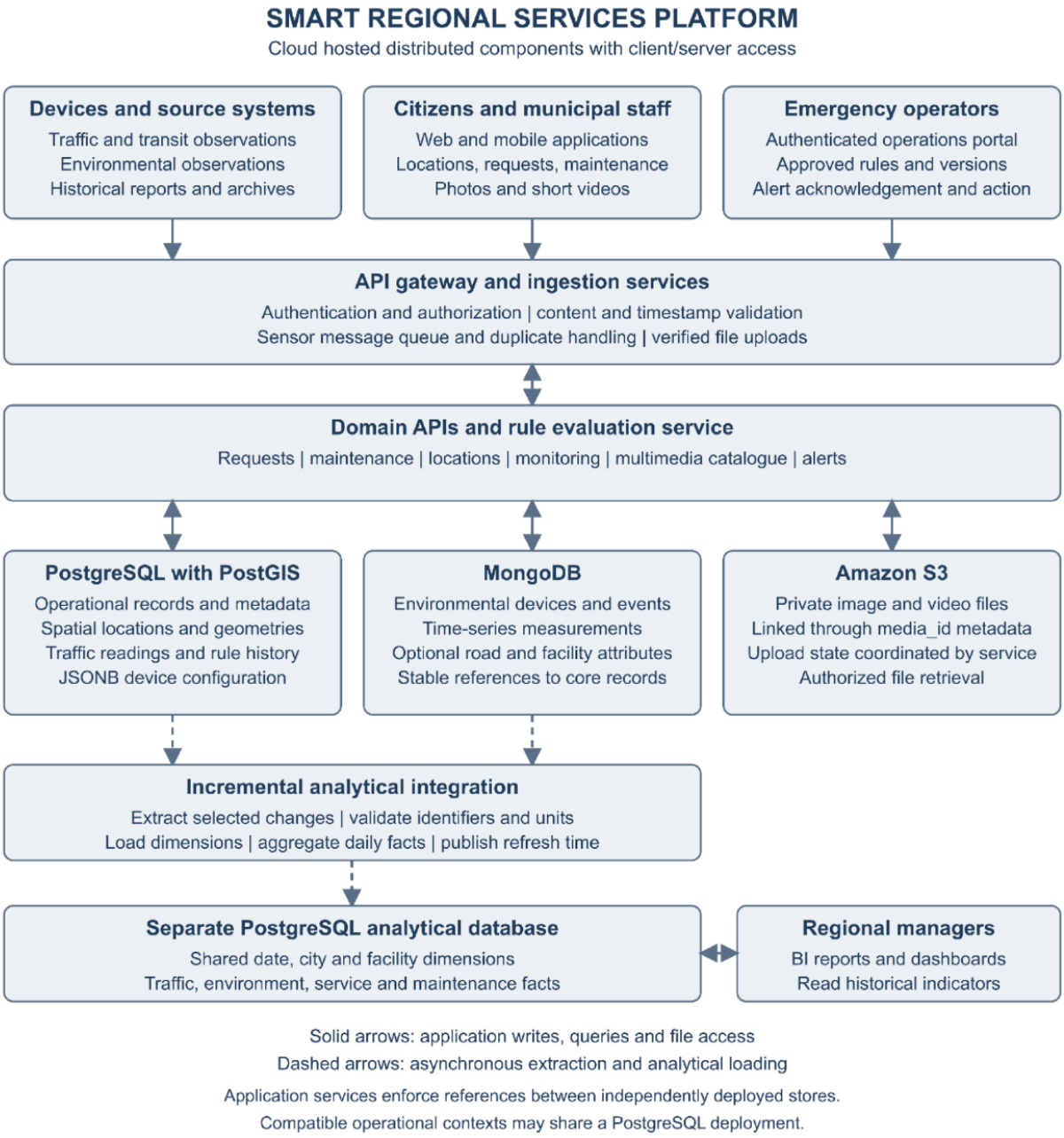
Operational decisions use current, actionable information. Municipal staff need open citizen requests, assigned maintenance work, and reliable status histories. Emergency operators need relevant observations, effective rules, and alerts awaiting acknowledgement. Transportation and environmental monitoring also support immediate assessments of traffic or hazardous conditions. These same records can support analytical decisions after consolidation: managers can compare congestion, seasonal environmental patterns, request resolution, facility expenditure, and infrastructure usage across cities. Consequently, most operational sources serve both purposes, but analytical queries should run against a separate reporting component rather than competing with service delivery.

The requirements table connects each source to the way it will be accessed and the performance risk that follows. Its database implications are initial design decisions, not measured guarantees. The assignment does not provide device counts, event volumes, file sizes, concurrent-user estimates, latency targets, availability targets, or retention periods. These values must be confirmed before capacity and service-level commitments are established. Access permissions must also distinguish citizens, municipal teams, emergency personnel, and regional managers. Shared identifiers and validated references must preserve links between locations,

requests, work orders, and evidence. This combination of workload analysis, specialized data handling, and explicit assumptions provides the basis for the architecture recommendation.

Part 2 – Database Architecture Recommendation

The overall architecture separates operational service delivery from historical analysis. Figure 2 shows the proposed components, user groups, and major data flows.



Note. Solid arrows show application access. Dashed arrows show asynchronous analytical integration. Component placement represents responsibilities and does not require one server per context.

Architecture Justification

We recommend a cloud-hosted distributed architecture with client/server access and polyglot persistence. Applications access domain APIs, and the APIs coordinate several storage components according to their workloads. This follows the storage-selection principle discussed by Fowler (2011): different uses of data can justify different persistence approaches. Distribution here refers to cooperating operational, document, object-storage, and analytical components. Compatible contexts and participating cities may share database deployments. This arrangement supports coordinated services while preserving clear data ownership.

Citizens use web or mobile applications to submit requests, attach evidence, and follow progress. Municipal staff use authenticated applications to manage locations, triage requests, and coordinate maintenance. Emergency operators review alerts and record response actions, while regional managers use reporting dashboards. An API gateway centralises application access. Services check authorization and validate submitted content before accepting changes. An ingestion service and message queue buffer device observations during downstream delays. Upload verification precedes evidence availability.

PostgreSQL supports operational transactions for requests, departments, maintenance, transportation devices and readings, multimedia metadata, and emergency rule history. PostGIS extends the location component with geometry storage and spatial queries. JSONB stores transportation device configuration where fields differ by sensor type. Related updates, such as a

request status and its history, are committed within the owning operational database. A context identifies ownership and responsibility; several compatible contexts may share one PostgreSQL deployment. Independent stores exchange stable identifiers through application-managed references, without cross-database foreign keys.

MongoDB supports environmental device documents, observed events, and a proposed time-series collection for measurements. Supplementary road and facility documents remain optional: their value must be demonstrated against the simpler alternative of keeping variable attributes in PostgreSQL JSONB. Amazon S3 stores images and short videos, while PostgreSQL retains their metadata and evidence links. The upload service coordinates these components through pending, available, and failed states. A rule evaluation service consumes relevant observations, retrieves an approved rule version, and records resulting alerts and actions. Notification delivery uses retries and deduplication; database triggers are not assumed to implement the rule engine.

A separate PostgreSQL analytical database supports historical and planning workloads. An incremental integration process extracts selected operational changes, validates identifiers and units, and loads dimensions before daily facts. Reports compare cities, periods, traffic, environmental conditions, services and costs. Dashboards display refresh times to disclose analytical lag. Only metadata and indicators enter analytics; evidence files remain in S3. Keeping analytical processing separate reduces competition between broad reporting scans and operational transactions.

Scalability comes from expanding ingestion workers and application instances independently, buffering event bursts, and evaluating time partitioning and workload-specific indexes where justified. Additional cities use shared identifiers and city-scoped permissions,

while new sources enter through documented ingestion contracts. Maintainability depends on clear context ownership, versioned APIs, schema and document validation, monitored integration jobs, and tested backup and restoration procedures. These choices introduce costs: more components require monitoring, and asynchronous integration creates temporary differences between stores. The implementation should begin with shared locations, operational PostgreSQL services, and the multimedia catalogue, then add specialized components where workload evidence supports them. Capacity, retention, recovery, and response-time targets remain assumptions to validate with stakeholders.

Schema / Context	Proposed Technology	Responsibility / Business Justification
ROAD_AND_FACILITY_LOCATIONS	PostgreSQL with PostGIS + MongoDB	PostGIS stores geometries and supports spatial queries; MongoDB stores flexible attributes. Shared identifiers link the records.
MULTIMEDIA	PostgreSQL + Amazon S3	PostgreSQL stores metadata and record links; S3 stores images and videos.
TRANSPORTATION_SENSORS	PostgreSQL	Store time-stamped readings, events, and sensor attributes, with JSONB for device configuration.
ENVIRONMENTAL_MONITORING	MongoDB	Store time-stamped measurements, events, and device attributes.
CITIZEN_SERVICE_REQUESTS	PostgreSQL	Manage requests, statuses, priorities, and responsible departments.
FACILITY_MAINTENANCE	PostgreSQL	Manage inspections, work orders, teams, costs, and completion status.
EMERGENCY_RULES_AND_RESPONSE	PostgreSQL + Rule Evaluation Service	Store rules, versions, and history; evaluate events and generate alerts.
HISTORICAL_PLANNING	Separate PostgreSQL Analytical Database	Consolidate data for reporting and regional planning.

Detailed schema documentation is provided in Appendix A, Sections A.1–A.8 and Tables A1–A49. Entity relationships are illustrated in Appendix B, Figures B1–B8. Open Appendix A. Open Appendix B.

Part 3 – Develop a Storage and Retrieval Strategy

We apply the Part 2 architecture to all eight data sources. The table connects each workload to its stored information, access pattern, storage organisation and performance risk. The accompanying eight scripts implement schema creation and CRUD, including cross-schema relationships, MongoDB documents and multimedia metadata.

Data Source	Data Stored	Creation / Update Frequency	Retrieval Method	Retrieval Frequency / Timing	Primary Storage Purpose	Storage / Retrieval Strategy	Performance Risk
ROAD_AND_FACILITY_LOCATIONS	Cities, roads, facilities, service zones; geometries and optional attributes.	On infrastructure changes; occasional attribute corrections.	By UUID, city, distance, or zone intersection.	Frequent map and operational lookups.	Operational processing.	PostGIS geometries; matching GiST indexes; city indexes. Optional MongoDB documents share UUIDs. See Appendix C, Section C.1.	Full spatial scans; mismatched index expressions; dangling cross-store references.
MULTIMEDIA	Images/videos in S3; SQL metadata, checksum, upload state and evidence links.	Per upload; state changes after verification; occasional corrections.	Metadata by media or business-record ID; authorised object retrieval by bucket/key.	On demand; older evidence retrieved occasionally.	Long-term archiving and evidence access.	S3 files plus PostgreSQL catalogue; PK/unique bucket-key; verify upload before availability. See Appendix C, Section C.2.	Large transfers delay access; database BLOB growth; metadata/object inconsistency.
TRANSPORTATION_SENSORS	Installations, sensors, readings, transit arrivals, captures and media links.	Readings every few seconds; configuration	By sensor and time range; join installation to road; transit by stop/time.	Frequent monitoring and historical extraction.	Operational processing.	PostgreSQL; sensor/time indexes; source-event deduplication; JSONB configuration. Evaluate partitioning as volume	Write amplification; unbounded history scans; duplicate ingestion.

Data Source	Data Stored	Creation / Update Frequency	Retrieval Method	Retrieval Frequency / Timing	Primary Storage Purpose	Storage / Retrieval Strategy	Performance Risk
		updated occasionally.				grows. See Appendix C, Section C.3.	
ENVIRONMENTAL_MONITORING	Device documents, metric/unit readings, quality flags and observed events.	Continuous readings; occasional configuration/ event updates.	By device, metric and time; devices by geographic proximity.	Near-real-time monitoring and historical trends.	Operational processing.	MongoDB regular device/event collections; time-series readings indexed by device/time; application validation. See Appendix C, Section C.4.	Oversized scans; poor bucketing; duplicate readings; unsupported time-series update patterns.

Storage and Retrieval Strategy Continued

Data Source	Data Stored	Creation / Update Frequency	Retrieval Method	Retrieval Frequency / Timing	Primary Storage Purpose	Storage / Retrieval Strategy	Performance Risk
CITIZEN_SERVICE_REQUESTS	Requests, departments, priorities, assignments, status history and media links.	Each citizen report; updates during triage and resolution.	By request UUID/tracking number; queues by city, status and department.	Frequent during service delivery.	Operational processing.	PostgreSQL transactions; PK/unique tracking number; queue index; row lock and atomic status/history changes. See Appendix C, Section C.5.	Slow queues; lost updates; inconsistent current state and history.
FACILITY_MAINTENANCE	Inspections, work orders, teams, costs, equipment usage	Per inspection/order; updated during work;	By facility, order, team, status or	Frequent operations; occasional facility-his	Operational processing.	PostgreSQL FKs across shared schemas; facility/time and status indexes; transactional	Slow joins; contention; orphaned costs;

Data Source	Data Stored	Creation / Update Frequency	Retrieval Method	Retrieval Frequency / Timing	Primary Storage Purpose	Storage / Retrieval Strategy	Performance Risk
	and evidence links.	costs as incurred.	period; costs by currency.	tory reviews.		progress history. See Appendix C, Section C.6.	incorrect totals across currencies.
EMERGENCY_RULES_AND_RESPONSE	Policies, immutable approved versions, alerts, actions and escalation history.	Rules change occasionally; alerts/actions arrive on events.	Effective approved rules by scope; alerts by status/time; history by alert ID.	Event-driven, with prompt queue retrieval.	Operational processing.	PostgreSQL versioned rules and unique evaluation keys; external evaluation service; indexed alert queue. See Appendix C, Section C.7.	Duplicate alerts; burst backlog; evaluating stale/unapproved rules.
HISTORICAL_PLANNING	Date/city/facility dimensions; daily traffic, environmental, service and cost facts.	Scheduled daily refresh; rerun affected dates for corrections.	By reporting period, city, road, category, facility, metric/unit or currency.	Scheduled dashboards and occasional historical analysis.	Analytical processing.	Separate PostgreSQL database; load dimensions first; unique fact grains and idempotent upserts. See Appendix C, Section C.8.	Reporting competes with operations if combined; duplicated facts; stale or incomparable aggregates.

Storage and Retrieval Justification

We selected all eight sources because the platform combines structured transactions, temporal observations, geographic searches, files and historical analysis. Each source requires a storage approach that matches how records arrive and how applications retrieve them. We retain the technologies chosen in Part 2: PostgreSQL for operational records, PostGIS for geographic data, MongoDB for environmental information and optional location attributes, S3 for files, and a separate PostgreSQL analytical database.

Locations provide shared identifiers for the other operational contexts. PostGIS stores points, road lines and service-zone polygons, while spatial indexes support nearby-facility and intersection queries. Optional MongoDB attribute documents reuse the relational UUID as their document identifier.

Citizen requests and maintenance records need reliable relationships and coordinated updates. Their schemas share the operational PostgreSQL database, allowing actual foreign keys between requests, departments, facilities, inspections and work orders. Composite references also check municipality or facility consistency where relevant. Status changes lock the current row and record the corresponding history within one transaction. Queue and facility-history indexes support common retrieval patterns, while monetary summaries remain separated by currency.

Transportation readings arrive much more often than device configurations change. We index sensor identifiers with observation timestamps and retrieve bounded time ranges, avoiding unnecessary scans of the complete history. Source-event uniqueness prevents repeated SQL ingestion. Monthly partitioning remains a volume-driven extension rather than an implemented requirement. Environmental readings use a MongoDB time-series collection, with regular collections for device configuration and observed events. Because time-series validation and

updates have restrictions, the script validates measurements before insertion and demonstrates updates only to series metadata.

Multimedia separates large file contents from searchable metadata. S3 stores the objects, while PostgreSQL records identifiers, storage keys, checksums and upload states. Files become available only after successful transfer and verification. Request, inspection and capture linking tables preserve the business context. PostgreSQL and S3 cannot commit atomically together, so failures require visible states, retries and reconciliation. The same limitation applies to references crossing PostgreSQL and MongoDB; an application check is not a distributed foreign key.

Emergency response stores approved rule versions and links each alert to the exact version evaluated. An external service performs evaluation; unique evaluation keys prevent duplicate alerts. Historical planning instead loads shared dimensions before daily facts in a separate database. Idempotent upserts preserve each fact grain, reporting dates use one documented timezone, and environmental units remain explicit. This separation protects operational work from broad reporting scans. Retention periods, ingestion capacity and latency targets require stakeholder validation and workload measurements before production deployment.

Implementation instructions and executable file links appear in Appendix C, Table C1; CRUD examples are in Sections C.1–C.8.

Part 4 – Index Selection and Validation

Table 1 presents indexing strategies for eight Smart Regional Services Platform retrieval scenarios. We reuse matching Part 3 indexes and propose targeted additions for an ordered citizen backlog, geometry containment, environmental threshold searches and an urgent alert queue. Each recommendation connects an access pattern with its retrieval benefit and maintenance cost.

Table 1

Recommended Indexing Strategies for the Smart Regional Services Platform

Retrieval scenario	Table / collection	Search condition / access pattern	Recommended index fields	Supported access type	Retrieval benefit	Trade-off / maintenance cost
1. Find open citizen requests in a city, showing creation date/time, oldest first.	service_requests (PostgreSQL) Query: Appendix D, Section D.1.	Match city_id and status IN ('submitted', 'triaged', 'assigned', 'in_progress'). Return created_at; ORDER BY created_at ASC, request_id ASC.	Propose partial B-tree (city_id, created_at, request_id) WHERE status IN the four active values. Existing request_queue_idx supports filtering with a separate sort.	Equality on city; active-status subset; chronological retrieval.	The partial index groups active requests by city in creation order and supports stable timestamp ties. It can avoid the global sort needed with the existing city/status/department/date index.	Add only if backlog retrieval is frequent and selective. Extra storage and writes accompany inserts/deletes and transitions into or out of active statuses. Retain the general queue index for its separate workload.
2. Retrieve one traffic sensor's	traffic_readings (PostgreSQL)	Match sensor_id; recorded_at >= :start_time AND	Reuse traffic_sensor_time_idx: composite	Equality; temporal range;	Locates one sensor's entries and scans the requested	Each incoming reading adds an index entry,

Retrieval scenario	Table / collection	Search condition / access pattern	Recommended index fields	Supported access type	Retrieval benefit	Trade-off / maintenance cost
readings during a selected period, newest first.	Query: Appendix D, Section D.2.	recorded_at < :end_time. ORDER BY recorded_at DESC.	B-tree (sensor_id, recorded_at DESC).	ordered retrieval.	time interval in descending order. Reduces irrelevant history scans and can avoid a separate sort.	increasing storage and ingestion work. Time partitioning remains an option to evaluate, rather than an implemented feature.
3. Find facilities in a selected zone, nearest to farthest from a selected or incident location.	facilities; service_zones (PostGIS) Query: Appendix D, Section D.3.	Match zone_id/city; ST_Covers(z.boundary, f.location). Calculate ST_Distance using geography and the selected point. Return metres; sort by distance, then facility_id.	Add GiST (location) on facilities. Reuse the service_zones primary-key B-tree (zone_id) to select the zone.	Equality; spatial containment; distance-ordered retrieval.	The zone key and proposed geometry GiST reduce containment candidates before exact checks. Distances are calculated for matching facility points, then explicitly sorted (PostGIS Project, n.d.-b, n.d.-c).	The geometry index adds storage and location-update work. Distance calculation and sorting still cost CPU/memory; the containment index does not remove that sort. Existing geography indexing serves other distance patterns.
4. Retrieve a facility's work orders created during the last two years, newest first.	work_orders (PostgreSQL) Query: Appendix D, Section D.4.	Match facility_id; created_at >= :as_of minus two years AND created_at <= :as_of. ORDER BY created_at	Reuse work_facility_time_idx: composite B-tree (facility_id, created_at DESC).	Equality; temporal range; ordered retrieval.	Narrows retrieval to one facility and the required creation-date interval. Can return matching work orders newest first	Consumes storage and requires maintenance for inserts, deletes and indexed-field changes. Completion-date or

Retrieval scenario	Table / collection	Search condition / access pattern	Recommended index fields	Supported access type	Retrieval benefit	Trade-off / maintenance cost
		DESC. Include all work-order statuses.			without a separate sort.	inspection-date searches require a separately matched access path.
5. Retrieve available multimedia evidence attached to one citizen request.	request_media; media_assets (PostgreSQL); files in S3 Query: Appendix D, Section D.5.	Match request_id; join on media_id; filter upload_status = 'available'. Use the returned bucket/key for authorised S3 retrieval.	Reuse primary-key B-trees: request_media (request_id, media_id) and media_assets (media_id). No additional index is required.	Equality; indexed join; availability filtering.	Finds attachment links by request and supports direct metadata lookups by media ID. Checks availability on matching assets before returning file references.	Existing key indexes consume storage and require maintenance on inserts/deletes. Their reuse adds no new index cost. S3 transfer time and metadata/object reconciliation remain separate concerns.
6. Find valid environmental readings above a threshold for one metric and unit in a city during a selected period.	environmental_readings (MongoDB time-series) Query: Appendix D, Section D.6.	Match metadata.city_id, metadata.metric and metadata.unit; recorded_at in [start, end); value > :threshold; quality_status = 'valid'.	Propose compound index: metadata.city_id: 1; metadata.metric: 1; metadata.unit: 1; recorded_at: -1. Retain device_time_idx for per-device retrieval.	Metadata equality; temporal range; threshold filtering.	Narrows candidate buckets by city, metric, unit and time before checking individual values and quality. Threshold and quality remain residual filters (MongoDB, Inc., n.d.-d).	An additional index increases bucket-index storage and ingestion work. Large time windows still examine many readings; confirm selectivity before adoption. It does not enforce reading uniqueness.

Retrieval scenario	Table / collection	Search condition / access pattern	Recommended index fields	Supported access type	Retrieval benefit	Trade-off / maintenance cost
7. Retrieve unacknowledged urgent emergency alerts, oldest first, for prompt operator attention.	alerts (PostgreSQL) Query: Appendix D, Section D.7.	status = 'open' AND priority = 'urgent'; ORDER BY triggered_at ASC LIMIT :page_size. Keep these two status/priority values explicit in the query.	Propose partial B-tree on (triggered_at) WHERE status = 'open' AND priority = 'urgent'. Existing alert_queue_idx supports the general status queue.	Equality on status/priority; ordered retrieval from a partial index.	If open urgent alerts are a small subset, indexes only that queue and supports oldest-first retrieval without scanning other priorities. The query must imply the index predicate (PostgreSQL Global Development Group, n.d.-e).	Adopt only for a frequent, selective urgent queue. Status/priority changes add or remove entries; bursts can reduce selectivity. The general queue index still needs maintenance.
8. Locate active environmental monitoring devices near an incident, nearest first.	monitoring_devices (MongoDB regular collection) Query: Appendix D, Section D.8.	Use location.$near with a GeoJSON incident point and $maxDistance in metres; filter status = 'active'; limit results. Coordinates use longitude, latitude.	Reuse device_location_geo: {location: '2dsphere'}. The index covers device positions, not the time-series readings.	Spatial proximity; distance-ordered retrieval; status filtering.	Supports spherical proximity search and nearest-first results, then retains active devices. Reuses a different spatial access method from PostGIS zone containment (MongoDB, Inc., n.d.-a, n.d.-e).	Consumes storage and requires maintenance when devices move or are inserted/deleted. Status is a residual filter; many nearby inactive devices can increase work. Missing locations have no spatial index entry.

Note. GiST = Generalized Search Tree; DESC = descending order; S3 = Amazon Simple Storage Service.

Index Design Rationale

Composite key order follows the query: equality fields lead the time range in scenarios 2 and 4. In scenario 1, the existing city/status queue index filters active requests but does not supply a global creation-date order. The proposed partial index leads with city and creation time and adds request_id for stable ties; adopt it when workload evidence justifies another index (PostgreSQL Global Development Group, n.d.-d, n.d.-e). Scenario 5 reuses primary-key indexes (PostgreSQL Global Development Group, n.d.-a). In scenario 3, GiST supports containment filtering, while geographic distance in metres is calculated and sorted separately. Neither query requires an index on every returned field.

Validate proposed additions with representative city sizes, time windows and write rates. Compare query plans, rows or buckets examined, sorting, index size and ingestion cost. Use PostgreSQL EXPLAIN (ANALYZE, BUFFERS) for read queries and MongoDB explain("executionStats"). Retain additions only when retrieval gains justify maintenance; broad result sets may favour scans. These are design recommendations, not measured performance results. The scenarios use operational stores; the final Redshift analytical recommendation requires its own physical design.

See Appendix D, Sections D.1–D.8, for the complete queries supporting the eight indexing scenarios.

Part 5 – Evaluate Storage and Indexing Trade-Offs

We compare three storage and indexing choices already introduced in Part 3. The table identifies their benefits, limitations, suitable workloads, and implications for growth and maintenance.

Recommended Strategy	What It Improves	Cost or Limitation	Most Suitable Workload	Impact on Scalability	Future Maintenance
Relational PostgreSQL with conventional and PostGIS spatial indexes	PKs/FKs preserve relationships among requests, departments, facilities and work orders. Transactions coordinate status/history updates. Indexes accelerate ID, status, date and location searches.	Indexes consume storage and add write overhead. Constraints, joins and concurrent transactions add processing and possible lock contention.	Mixed operational workloads; spatial searches mainly favour reads.	Targeted indexes reduce scans as records grow. Higher write volumes still require capacity and concurrency planning; an index alone does not distribute writes.	Review query plans and index usage; maintain statistics, valid geometries and consistent coordinate systems; plan schema migrations.
MongoDB time-series collections for environmental readings	Organises observations by device and time, supporting efficient storage and bounded temporal retrieval. Regular collections retain device configurations and observed events.	Time-series updates and validation are restricted. PostgreSQL identifiers are logical references requiring application checks, not cross-database FKs.	Write-heavy ingestion with time-based reads; measurements seldom change.	Growing observation volumes can use appropriate retention and distribution. Shard keys must suit device/time access and avoid uneven ingestion.	Validate incoming readings; handle duplicates and corrections; review indexes, retention and distribution; reconcile external references.
S3 multimedia files with PostgreSQL metadata	Keeps images/videos outside operational tables while retaining searchable metadata and	Object uploads and database updates can fail independently. Downloads add transfer latency and	Mixed uploads, metadata searches and file retrieval;	Object storage grows independently of relational processing. Metadata queries and network throughput	Track upload states, verify checksums, control access and coordinate retention; reconcile missing

Recommended Strategy	What It Improves	Cost or Limitation	Most Suitable Workload	Impact on Scalability	Future Maintenance
	links to requests, inspections and captures.	possible request/transfer costs.	older evidence is accessed occasionally.	remain separate capacity concerns.	objects and unreferenced files.

Evaluation of Storage and Indexing Choices

We evaluate three choices from the Part 3 architecture: indexed relational PostgreSQL, MongoDB time-series storage, and S3 files linked to PostgreSQL metadata. Effectiveness depends on workload and access patterns. Our comparison considers retrieval, write and coordination costs, scalability and maintenance.

Relational PostgreSQL suits citizen requests and maintenance because primary and foreign keys preserve relationships, while transactions coordinate changes such as a current status and its history (PostgreSQL Global Development Group, n.d.-a, n.d.-f). Conventional indexes support identifier, status and date filters; PostGIS indexes support location searches using compatible functions (PostGIS Project, n.d.-a). However, indexes occupy space and must be updated when indexed records change, adding write overhead (PostgreSQL Global Development Group, n.d.-b). We favour indexes aligned with actual queries. As volumes grow, query-plan reviews, statistics maintenance, geometry validation and controlled schema migrations remain necessary.

MongoDB time-series collections suit environmental observations arriving continuously from devices. Time and stable metadata organise the series for efficient storage (MongoDB, Inc., n.d.-b). We use device-and-time retrieval to limit the observations examined. The trade-off is reduced flexibility for correcting measurements: time-series collections restrict updates and do not support schema validation rules (MongoDB, Inc., n.d.-c). Consequently, our ingestion service validates readings before insertion, while regular collections retain device configuration and observed events. This is appropriate for a write-heavy workload where measurements rarely change. Retention and suitable shard keys can support growth, but distribution requires planning around ingestion and query patterns. Maintenance also includes duplicate handling, correction procedures and reconciliation of city or facility identifiers owned by PostgreSQL.

S3 separates multimedia objects from searchable relational metadata. Its object-storage and management capabilities support independent growth of the file repository (Amazon Web Services, n.d.-d). Applications retrieve metadata by media or business-record identifier before requesting the object. This fits mixed uploads, searches and downloads, while avoiding large file contents in operational tables. The limitation is coordination across stores. AWS describes the general dual-write risk: one operation can succeed while another fails, creating inconsistent state (Amazon Web Services, n.d.-a). Applied to our file-and-catalogue workflow, this motivates pending, available and failed upload states, checksum verification, retries and reconciliation. Metadata capacity, download throughput and transfer costs must still be monitored.

The strategies address complementary workloads. PostgreSQL prioritises related-record integrity and selective operational retrieval; MongoDB prioritises ongoing temporal ingestion; S3 separates file growth from relational processing. Future maintenance should track query performance, write overhead, storage growth and failed integrations. Capacity and retention targets require workload evidence.

Part 6 – Recommend Specialized Database Models

We recommend approaches for all seven data categories. The table explains where relational storage needs an extension, additional organisation or a complementary service, and relates each choice to platform operations and its principal risk.

Data Category	Why a Basic Relational Table Needs Extension or Organisation	Recommended Model or Approach	Queries or Operations Supported	Contribution to Platform Goals	Limitation or Risk
Spatial location data	Separate latitude/longitude values do not provide geometry types, coordinate-system handling or indexed spatial relationships.	PostgreSQL with PostGIS; point, line and polygon geometries; GiST spatial indexes.	Facilities within a distance; roads intersecting zones; point-in-zone searches.	Locate incidents, select nearby facilities and coordinate services geographically.	Invalid geometries, inconsistent coordinate systems or unsuitable query expressions can produce incorrect or slow results.
Multimedia evidence	Tables can hold binary data, but storing large file contents with transactional records does not provide the chosen object-storage delivery and lifecycle approach.	S3 object storage plus PostgreSQL media metadata and evidence-link tables.	Upload/retrieve an object; search evidence by request, inspection or capture ID; verify checksum and availability.	Keep evidence accessible and traceable without enlarging operational tables with file contents.	Uploads and metadata updates can fail independently; access, retention and reference reconciliation must be coordinated.
Time-series or temporal sensor data	A relational table can store timestamps, but continuous ingestion requires suitable time organisation, bounded queries and duplicate handling.	MongoDB time-series collections for environmental readings; PostgreSQL sensor/time indexes for transportation. Evaluate	Device readings over a period; metric/unit aggregates; traffic trends and transit-arrival comparisons.	Support monitoring and provide temporal observations for alerts and planning.	Late or repeated observations distort results; MongoDB time-series updates are restricted and cross-store snapshots may differ.

Data Category	Why a Basic Relational Table Needs Extension or Organisation	Recommended Model or Approach	Queries or Operations Supported	Contribution to Platform Goals	Limitation or Risk
		time partitioning if volume warrants it.			
Hierarchical or semi-structured service requests	A flat request row cannot express department ancestry by itself; variable fields need a deliberate schema and validation policy.	Normalised PostgreSQL requests and a self-referencing department hierarchy. Recursive SQL traverses it; JSONB is an optional extension for justified variable attributes.	Find descendant departments; list their assigned requests; retrieve status history; optionally filter validated JSONB attributes.	Preserve accountable ownership and allow variation without abandoning relational integrity.	Cycles and deep recursion need control. Optional JSONB requires field validation and an explicit schema migration.

Specialized Database Models Continued

Data Category	Why a Basic Relational Table Needs Extension or Organisation	Recommended Model or Approach	Queries or Operations Supported	Contribution to Platform Goals	Limitation or Risk
Active database behaviour	Ordinary stored rows do not automatically execute a response when another row changes.	PostgreSQL event-condition-action triggers for local validation and invariants; proposed outbox/worker for asynchronous notifications if required.	On INSERT/UPDATE, validate camera type, department hierarchy or work-order completion; reject invalid changes within the transaction.	Apply shared local rules consistently across write paths.	Expensive or cascading triggers add latency and hidden coupling. Asynchronous notifications require retries and delivery monitoring.
Rule-based emergency response logic	Storing a threshold is insufficient to select an effective policy, evaluate an event and govern response actions.	An external rule-evaluation service; approved versioned JSONB conditions in PostgreSQL; alerts linked to exact rule versions.	Select active/effective rules; evaluate an observation; deduplicate alerts; record acknowledgement and escalation.	Make emergency decisions traceable and reproducible under approved policies.	Conflicting rules, stale versions, event bursts or repeated delivery require governance, retries and idempotency.
Historical planning data	Operational tables can answer reports, but repeated broad joins compete with service activity and lack a	A dimensional star-schema approach in a separate PostgreSQL analytical database; date/city/facility	Compare cities and periods; aggregate traffic, environmental metrics, service performance and maintenance cost.	Provide consistent planning indicators while managing reporting separately from operations.	Refresh lag and inconsistent units or grains can mislead. Separate databases still share resources if

Data Category	Why a Basic Relational Table Needs Extension or Organisation	Recommended Model or Approach	Queries or Operations Supported	Contribution to Platform Goals	Limitation or Risk
	shared analytical grain.	dimensions and daily facts.			deployed on the same server.

Justification of Specialized Database Approaches

We recommend specialised approaches for all seven categories while retaining the Part 2 architecture. Relational storage remains appropriate for accountable operational records; extensions and complementary models address geometry, files, time, hierarchy, automatic reactions, governed rules and analytical summaries. Elmasri and Navathe (2016) provide the conceptual foundation for distinguishing these advanced models.

PostGIS adds geometries and spatial operations to PostgreSQL, enabling distance, intersection and containment queries. Index-aware operations can use GiST indexes to reduce candidate comparisons (PostGIS Project, n.d.-a). This supports incident localisation and service-zone coordination. Valid geometries, consistent coordinate systems and appropriate distance units remain essential; otherwise, even a fast query can return misleading results.

Multimedia uses S3 for object contents and PostgreSQL for searchable metadata and business links. Large files remain outside operational tables, while identifiers connect evidence to requests, inspections and captures. Object retrieval and lifecycle management support this separation (Amazon Web Services, n.d.-d). However, a database change and file transfer can fail independently. Pending states, verification and reconciliation remain necessary. Kleppmann (2017) explains why independent stores require recovery from partial failures.

Environmental readings use MongoDB time-series collections, while transportation retains PostgreSQL tables with sensor/time indexes. MongoDB provides storage organised for time-series measurements, but restricts updates and schema validation (MongoDB, Inc., n.d.-b, n.d.-c). Queries use bounded periods and preserve metric/unit distinctions. Late arrivals, duplicates and corrections need explicit policies. Transportation time partitioning should be evaluated if sustained growth justifies the additional maintenance.

Service requests remain relational because their assignments and histories require stable relationships. Department ancestry uses a self-referencing hierarchy and recursive SQL; PostgreSQL supports recursive common table expressions (PostgreSQL Global Development Group, n.d.-h). If variable request attributes emerge, JSONB could supplement typed columns, with validation and indexing (PostgreSQL Global Development Group, n.d.-c). Adding it would require a reviewed schema migration. Hierarchy-cycle checks and controlled recursion prevent invalid organisational paths.

We use PostgreSQL triggers for local conditions such as camera classification, department-cycle prevention and completion validation. These mechanisms follow the event-condition-action approach described by Elmasri and Navathe (2016) and execute within the database write path (PostgreSQL Global Development Group, n.d.-g). They support consistent validation, but excessive trigger work increases latency and coupling. External notifications belong to a worker; a transactional outbox would require its own schema and delivery worker.

An evaluation service selects an approved, effective rule version, checks incoming observations and records alerts referencing that version. PostgreSQL preserves the governed conditions and response history, while the service performs evaluation. This separation makes policy changes auditable and supports replay and deduplication. Rule conflicts, stale versions and event bursts require approval controls, idempotency and monitored processing.

Historical planning uses dimensional modelling: shared dimensions describe dates, cities and facilities, while facts retain explicit daily grains. Kimball and Ross (2013) provide the basis for this analytical organisation. Period comparisons and grouped indicators support regional decisions without repeatedly rebuilding meaning from operational joins. Refresh lag, unit

compatibility and dimension changes remain risks. A separate analytical database enables distinct reporting management, but independent resource allocation is still required when operational and analytical databases share infrastructure.

Part 7 – Compare Contemporary Data Processing Paradigms

We compare all eight paradigms listed in the assignment. Each recommendation identifies its role in the final architecture, the sources it supports, suitable workloads and important limitations. Storage and processing responsibilities overlap, but remain distinct.

Paradigm and Definition	Platform Data Supported	Workloads It Handles Well	Workloads It Handles Poorly	Recommendation for the Final Platform
Relational database systems — typed tables, keys, constraints, joins and transactions.	Citizen requests, departments, maintenance, transport records, multimedia metadata and emergency history.	Consistent record updates, identifier/status lookups, related-record retrieval and transactional workflows.	Large file delivery and broad historical scans competing with live service transactions.	Yes. PostgreSQL remains the main operational store; isolate analytical workloads and keep file contents in S3.
Spatial database systems — geometry-aware storage, indexing and spatial operations.	Roads, facilities, service zones, monitoring points and incident locations.	Distance, intersection, containment and spatial joins using appropriate geometries and indexes.	Non-spatial requests gain little from spatial indexes; spatial functionality alone does not manage files or event delivery.	Yes. Use PostGIS within PostgreSQL, with valid geometries, consistent coordinate systems and index-aware queries.
Document or semi-structured stores — records represented as nested documents with controlled variation.	Environmental device configuration/events and optional location attributes; transportation configuration uses PostgreSQL JSONB.	Retrieving a complete device document and managing attributes that vary by device type.	Cross-store referential integrity and relationship-heavy retrieval requiring many joins or application checks.	Yes, within a defined scope. MongoDB owns environmental documents; retain operational relationships in PostgreSQL and validate shared IDs.
Multimedia management — object/file storage combined with a searchable metadata catalogue.	Images, videos and their links to citizen requests, inspections and traffic captures.	Uploading/retrieving large objects and finding evidence through business-record metadata.	Object storage alone does not perform relational joins or semantic content searches; these require	Yes. S3 holds files; PostgreSQL holds metadata and evidence links. Coordinate upload states, integrity, access and retention.

Paradigm and Definition	Platform Data Supported	Workloads It Handles Well	Workloads It Handles Poorly	Recommendation for the Final Platform
			metadata processing or extra services.	

Contemporary Data Processing Paradigms Continued

Paradigm and Definition	Platform Data Supported	Workloads It Handles Well	Workloads It Handles Poorly	Recommendation for the Final Platform
Time-series processing — observations organised around timestamps and stable series identifiers.	Environmental measurements, traffic readings and temporal transit observations.	Append-heavy ingestion, device/time-range queries and aggregates over comparable metrics and units.	Frequent historical rewrites and complex entity relationships; late or duplicate observations need explicit handling.	Yes. MongoDB time-series collections for environmental readings; indexed PostgreSQL temporal tables for transportation, consistent with Part 2.
Rule-based processing — evaluate governed conditions against facts/events and record resulting decisions.	Emergency policies, triggering observations, alerts, response actions and escalations.	Repeatable threshold/policy evaluation, approved rule-version selection and traceable responses.	Bulk archival retrieval or broad analytical aggregation; complex or conflicting conditions can delay event evaluation.	Yes. An external evaluation service uses versioned PostgreSQL rules and records deduplicated alerts; local triggers retain local validation duties.
Data warehousing and analytical processing — integrated facts and dimensions organised for reporting.	Historical traffic/environment summaries, service performance and maintenance costs.	Period/city comparisons, grouped indicators and planning queries across consistent reporting dimensions.	Immediate operational updates and current alert decisions; refreshed facts may lag behind source events.	Yes. The final Part 8 recommendation refines the analytical store to Redshift, supported by S3, Glue and EMR. Retain declared fact grains and display the latest successful refresh time.

Paradigm and Definition	Platform Data Supported	Workloads It Handles Well	Workloads It Handles Poorly	Recommendation for the Final Platform
Streaming or event-based processing — continuously process published observations as they arrive.	Incoming traffic and environmental events; inputs to rule evaluation and asynchronous integration.	Decoupled producers/consumers, buffering bursts, prompt event reactions and replay when the chosen event infrastructure supports it.	Broad historical ad hoc queries without retained data/views, or atomic changes across independent consumers.	Yes. Use the Part 2 queue and ingestion/evaluation workers. Choose the broker after workload assessment; manage retries, ordering and duplicate delivery.

Comparison and Recommended Roles

We compare eight paradigms because the platform combines accountable transactions, specialised information, historical reporting and continuous observations. They address different responsibilities and can coexist within one architecture. The recommendation retains the Part 2 technologies and distinguishes data storage models from processing approaches. Effectiveness depends on access patterns and consistency needs, rather than using one system for every workload (Kleppmann, 2017).

Relational processing remains central to requests, maintenance and operational histories. Keys and transactions support related updates, such as changing a request status together with its history (PostgreSQL Global Development Group, n.d.-a, n.d.-f). This fits mixed operational workloads, but long regional aggregations can compete with service activity. Spatial processing complements this model: PostGIS supports geographic relationships and index-aware searches (PostGIS Project, n.d.-a). Nearby-facility and zone-intersection queries benefit, whereas ordinary status lookups still depend on conventional relational indexes.

Document processing suits heterogeneous environmental device configurations and observed events. Retrieving a device and its nested settings can be straightforward, but links to PostgreSQL cities and facilities still need application validation. We therefore keep this scope bounded. Time-series processing addresses a separate concern: repeated observations identified by device and timestamp. Environmental readings use MongoDB time-series collections; transportation retains indexed PostgreSQL temporal tables. This supports append-heavy ingestion and period queries, while update restrictions, late arrivals and duplicate delivery complicate corrections (MongoDB, Inc., n.d.-b, n.d.-c).

Multimedia management separates object contents from structured metadata. S3 supports file storage and retrieval, while PostgreSQL links evidence to business records

(Amazon Web Services, n.d.-d). This suits uploads, downloads and metadata searches, but does not itself provide semantic image search. Independent file and catalogue failures require verification and reconciliation. Its responsibility differs from the relational store even though both participate in one evidence workflow.

Rule-based processing applies approved conditions to incoming observations and records the exact rule version behind each alert. This provides traceable decisions, but requires conflict checks and protection against repeated evaluations. Streaming processing transports and processes those observations continuously. Apache Kafka documentation illustrates producers publishing events and consumers processing subscribed streams
(Apache Software Foundation, n.d.). Our recommendation adopts the event-processing paradigm through the queue and workers already proposed, while leaving the broker choice open. Buffering can absorb bursts, but ordering, replay, consumer failures and duplicate handling remain design responsibilities (Kleppmann, 2017).

Analytical processing consolidates selected operational changes into shared dimensions and daily facts. This supports planning comparisons across cities and periods, following dimensional modelling principles (Kimball & Ross, 2013). It is less appropriate for immediate case updates or current emergency decisions because refreshes can lag. Streaming supports prompt reactions; the warehouse supports broader historical interpretation. Retained source observations and documented reporting grains connect the two.

We recommend all eight paradigms within these defined roles. Their combination supports operational consistency, specialised retrieval and timely monitoring while preserving an analytical view for planning. Maintenance must coordinate identifiers, units, timestamps, access controls and integration failures across the participating components.

Part 8 – Final Database Strategy Recommendation

We recommend a cloud-hosted platform combining PostgreSQL, PostGIS and MongoDB for operations with an AWS analytical layer using S3, Glue, EMR and Redshift. This refines the earlier PostgreSQL analytical database proposal. PostgreSQL remains the operational foundation; shared schemas preserve relationships without requiring a server per department. Complementary technologies should follow workload needs and integration costs (Kleppmann, 2017).

Operational schemas should share stable UUIDs for cities, roads, facilities, requests, work orders and evidence. Primary keys, unique constraints and foreign keys preserve relationships within PostgreSQL. Transactions coordinate related changes, including current status and its history (PostgreSQL Global Development Group, n.d.-a, n.d.-f). Services validate state transitions and enforce city-scoped access. References crossing PostgreSQL, MongoDB and analytical stores require validated integration contracts; they are not cross-database foreign keys.

Indexes should match recurring searches: primary and unique indexes for identifiers and tracking numbers, composite indexes for city/status/department queues, sensor/time indexes for bounded observations, and facility/time indexes for maintenance histories. PostGIS GiST indexes must match spatial expressions for proximity, containment and intersection searches (PostGIS Project, n.d.-a). Use bounded queries and pagination. Additional indexes require representative query plans because they consume storage and increase write costs (PostgreSQL Global Development Group, n.d.-b).

PostGIS manages geometry and coordinate systems. Normalised tables preserve request and maintenance relationships; recursive queries support department hierarchies, with JSONB reserved for justified variable attributes. MongoDB regular collections hold environmental devices and events; time-series collections hold measurements by device and time. Ingestion

validates metrics, units and quality flags and handles repeated deliveries. Time-series restrictions require explicit correction policies (MongoDB, Inc., n.d.-b, n.d.-c). Transportation observations retain their PostgreSQL design.

S3 stores multimedia contents; PostgreSQL stores metadata, checksums, upload states and evidence links. Applications search the catalogue before authorised downloads. Pending, available and failed states, verification and reconciliation coordinate independent file and database operations (Amazon Web Services, n.d.-d). A separate evaluation service selects approved, effective rule versions and records deduplicated alerts and response actions. Database triggers enforce local invariants; queues and workers support ingestion and reactions.

For analytics, incremental PostgreSQL and MongoDB extracts enter a dedicated S3 data lake, separate from evidence files. Retain raw inputs and publish validated Parquet datasets partitioned by useful dates and cities. Glue catalogues schemas and performs routine integration and cleansing (Amazon Web Services, n.d.-e). EMR handles heavier Spark joins, sensor aggregation and reprocessing; assign transformations to Glue or EMR to avoid duplicate work (Amazon Web Services, n.d.-b). Redshift serves SQL reporting over curated dimensions and facts (Amazon Web Services, n.d.-c; Kimball & Ross, 2013). Preserve source identifiers and display refresh times.

Scale ingestion workers independently, buffer bursts through queues and separate object growth from metadata processing. Evaluate environmental distribution and transportation time partitioning against measured volumes. Scale S3, EMR and Redshift independently from operational workloads. Scheduled jobs, suitable file sizes and measured query plans control costs. Shared identifiers, explicit fact grains and unit/currency separation preserve meaning across cities.

The first trade-off is workload-specific storage versus integration complexity: specialised components improve suitability but introduce independent failures, reconciliation, pipeline delays, cloud costs and expertise requirements. Clear ownership, durable identifiers, idempotent consumers and monitored recovery mitigate these costs. The second is retrieval performance versus write and maintenance overhead. Indexes, summaries and partitions accelerate selected queries but increase storage and refresh work. Selective indexing, bounded queries, retention policies and measured refresh schedules balance these demands. Validate both trade-offs against representative workloads before committing capacity.

Prioritise shared locations, operational PostgreSQL, request and maintenance workflows, and reliable S3 evidence uploads. Establish access controls, tested backup restoration and monitoring alongside these foundations. Next add durable sensor ingestion, MongoDB environmental storage and governed emergency evaluation. Then establish S3 landing zones, the Glue catalogue, essential transformations and a small Redshift reporting model. Introduce EMR as heavier workloads emerge after identifiers, units and histories are reliable. Optional location documents, advanced content search, sharding and partitioning should follow demonstrated needs. Review response times, write throughput, storage growth, pipeline failures and recovery results regularly to guide subsequent investment and confirm that additional cities receive consistent service quality. Before expanding the platform, confirm retention periods, recovery objectives and acceptable refresh delays with stakeholders. Test restoration across participating stores and document responsibility for correcting inconsistent references, delayed observations and failed uploads during routine operational incident handling.

Conclusion

The platform combines PostgreSQL and PostGIS for operational and spatial data, MongoDB for environmental documents and measurements, and S3 for multimedia. Shared identifiers connect these stores, while workload-specific indexes improve retrieval with manageable maintenance costs. We recommend separating analytics from operational services through an S3 data lake, Glue for cataloguing and integration, EMR for complex transformations, and Redshift for reporting. Implementation should prioritise shared locations and transactional workflows, followed by sensors, emergency processing and analytics. Query plans, access controls, data quality checks and tested recovery procedures should validate the design before deployment and support reliable services across cities over time.

Group Contribution Statement

All four group members contributed equally and worked collaboratively throughout the assignment. The work was organised into epics, features and tasks using a Kanban approach to coordinate responsibilities and track progress. Members collaborated on developing, reviewing and integrating the report's sections into a coherent final submission. Fabio Augusto De Oliveira Dutra, Gumashtha Devangiri Pandukabhaya Punchi Wedikkarage Don, Jose Vidal Asencio Santos, and Mehrthur Luís Lemos da Silva each acted as a researcher and developer.

AI Usage Statement

Generative AI tools assisted with checking word counts, refining language, and structuring content. The group reviewed the AI-assisted content, checked technical claims against the cited sources, and revised the wording to ensure accuracy and originality.

References

Amazon Web Services. (n.d.-a). *Transactional outbox pattern*. https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Amazon Web Services. (n.d.-b). *What is Amazon EMR?* https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-what-is-emr.html

Amazon Web Services. (n.d.-c). *What is Amazon Redshift?* https://docs.aws.amazon.com/redshift/latest/mgmt/welcome.html

Amazon Web Services. (n.d.-d). *What is Amazon S3?* https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

Amazon Web Services. (n.d.-e). *What is AWS Glue?* https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html

Apache Software Foundation. (n.d.). *Introduction*. https://kafka.apache.org/intro/

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Fowler, M. (2011, November 16). *Polyglot persistence*. https://martinfowler.com/bliki/PolyglotPersistence.html

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

mehrthur-teste. (n.d.). *Public services platform serving multiple cities* [Source code]. GitHub. https://github.com/mehrthur-teste/public-services-platform-serving-multiple-cities

MongoDB, Inc. (n.d.-a). *$near (query predicate operator)*. https://www.mongodb.com/docs/v8.0/reference/operator/query/near/

MongoDB, Inc. (n.d.-b). *Time series*. https://www.mongodb.com/docs/v8.0/core/timeseries-collections/

MongoDB, Inc. (n.d.-c). *Time series collection limitations*. https://www.mongodb.com/docs/v8.0/core/timeseries/timeseries-limitations/

MongoDB, Inc. (n.d.-d). *Time series indexes*. https://www.mongodb.com/docs/v8.0/core/timeseries/timeseries-index/

MongoDB, Inc. (n.d.-e). *2dsphere indexes*. https://www.mongodb.com/docs/v8.0/core/indexes/index-types/geospatial/2dsphere/

PostGIS Project. (n.d.-a). *How do I use spatial indexes?* https://postgis.net/documentation/faq/spatial-indexes/

PostGIS Project. (n.d.-b). *ST_Covers*. https://postgis.net/docs/ST_Covers.html

PostGIS Project. (n.d.-c). *ST_Distance*. https://postgis.net/docs/ST_Distance.html

PostgreSQL Global Development Group. (n.d.-a). *Constraints*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ddl-constraints.html

PostgreSQL Global Development Group. (n.d.-b). *Introduction*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/indexes-intro.html

PostgreSQL Global Development Group. (n.d.-c). *JSON types*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/datatype-json.html

PostgreSQL Global Development Group. (n.d.-d). *Multicolumn indexes*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/indexes-multicolumn.html

PostgreSQL Global Development Group. (n.d.-e). *Partial indexes*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/indexes-partial.html

PostgreSQL Global Development Group. (n.d.-f). *Transactions*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/tutorial-transactions.html

PostgreSQL Global Development Group. (n.d.-g). *Triggers*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/triggers.html

PostgreSQL Global Development Group. (n.d.-h). *WITH queries (common table expressions)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/queries-with.html

Vazquez, C. E., & Simões, G. S. (2016). *Engenharia de requisitos*. Brasport.

Appendix A

Schemas and Data Dictionary

The following material preserves the detailed workflows, schema catalogue, field definitions and validation rules supporting Part 2. Sections A.1–A.8 identify the eight data sources; each detailed table has an A-prefixed number.

The order below begins with shared foundations and progresses to the processes that depend on them. Each context has a defined responsibility, workflow, and data ownership. Contexts may share technologies without requiring a separate physical database for each one.

Table A1

Order	Context	Responsibility	Workflow	Dependencies
1	**ROAD_AND_FACILITY_LOCATIONS**	Maintain cities, roads, public buildings, parks, bus stops, and service zones.	Municipal staff create or update a location → the system validates the information → stores geometry and attributes → makes the records available to other services.	Foundational context, independent of the other contexts presented.
2	**MULTIMEDIA**	Receive and catalogue images, videos, and, as a proposed extension, audio from different channels.	Source uploads a file → service validates it → stores it in S3 → records metadata → returns an identifier → originating context links the evidence to its record.	A file can exist before being linked to a request, inspection, or capture event.
3	**TRANSPORTATION_SENSORS**	Register devices and record speed, road occupancy, congestion, and	Sensor sends a reading → ingestion validates the device, timestamp, and content → stores the reading → updates monitoring views → forwards	ROAD_AND_FACILITY_LOCATIONS; MULTIMEDIA when images or

Order	Context	Responsibility	Workflow	Dependencies
		public transit events.	relevant events for rule evaluation. Camera captures are linked to multimedia evidence.	videos are involved.
4	**ENVIRONMENTAL_MONITORING**	Record air quality, temperature, noise, water levels, and environmental events.	Device sends a measurement → ingestion validates it → stores it → updates indicators → forwards relevant conditions for rule evaluation.	ROAD_AND_FACILITY_LOCATIONS; MULTIMEDIA when supporting evidence is available.
5	**CITIZEN_SERVICE_REQUESTS**	Register reported problems and support service tracking.	Citizen provides a category, description, and location → optionally attaches evidence → receives a tracking number → staff triage the request → assign responsibility → record progress → resolve and close it.	ROAD_AND_FACILITY_LOCATIONS; MULTIMEDIA for optional attachments. May generate a maintenance demand.
6	**FACILITY_MAINTENANCE**	Plan and track inspections, work orders, teams, equipment, and costs.	Inspection, service request, or preventive schedule generates a demand → coordinator creates a work order → assigns priority and team → team performs the work and records activities, costs, and evidence → supervisor verifies and closes the order → originating request is updated, where applicable.	ROAD_AND_FACILITY_LOCATIONS; MULTIMEDIA for evidence; CITIZEN_SERVICE_REQUESTS when work originates from a reported issue.
7	**EMERGENCY_RULES_AND_RESPONSE**	Define risk conditions and coordinate alerts, priorities, and escalations.	Authorized staff create and approve a rule → service receives an event → evaluates conditions and the affected zone → records an alert and rule version → notifies responsible personnel → tracks acknowledgement, action, escalation, and closure.	ROAD_AND_FACILITY_LOCATIONS and event sources, including TRANSPORTATION_SENSORS, ENVIRONMENTAL_MONITORING, and

Order	Context	Responsibility	Workflow	Dependencies
				CITIZEN_SERVICE_REQUESTS.
8	**HISTORICAL_PLANNING**	Consolidate information to analyse trends and support regional decisions.	Integration process extracts operational data → validates, standardizes, and aggregates it → loads the analytical database → produces indicators → managers compare periods and locations → plan resources, maintenance, transportation, and events.	Selected data from the other contexts.

Organization of data

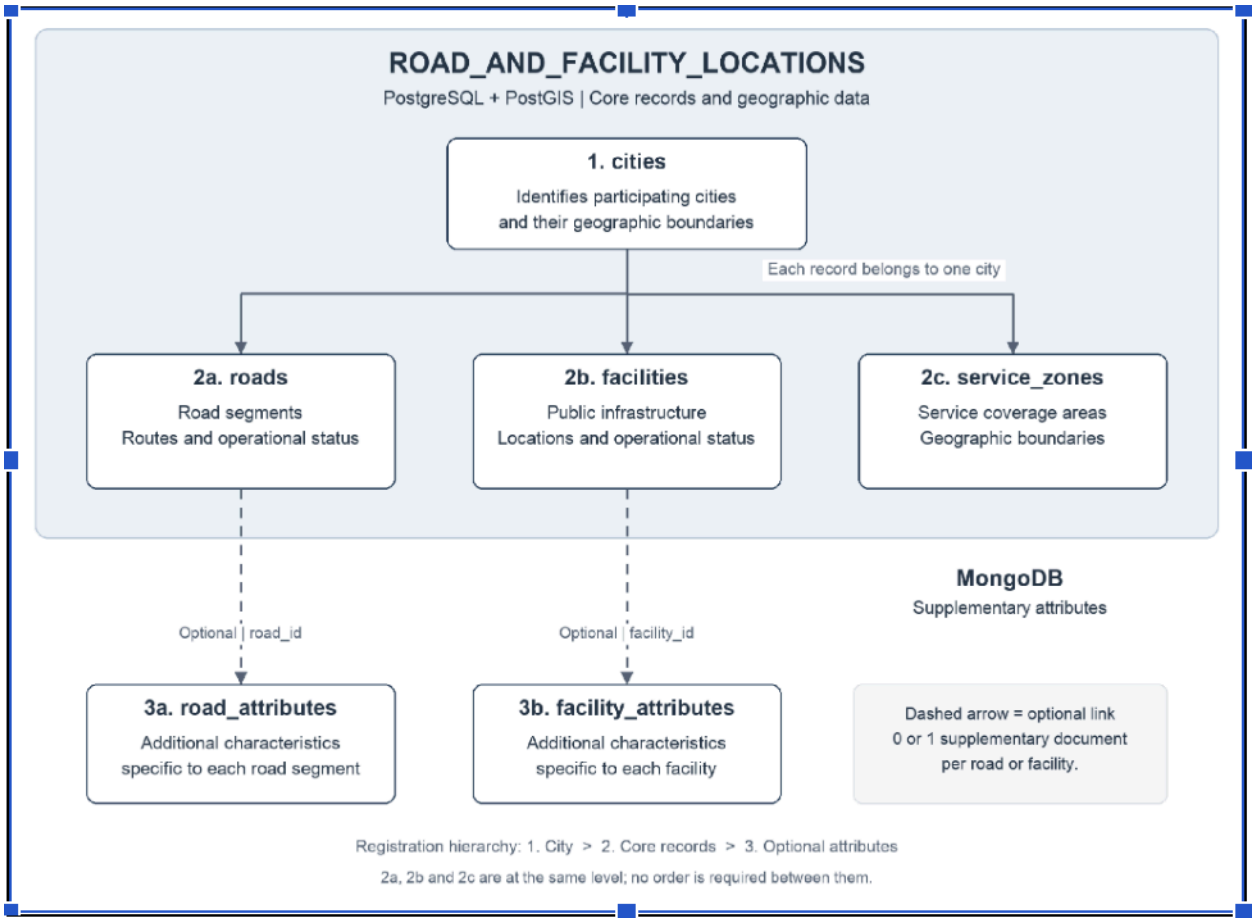
Table A2

Context / Schema	Main Proposed Tables or Collections	Proposed Technology
ROAD_AND_FACILITY_LOCATIONS	Tables: cities, roads, facilities, service_zones. Collections: road_attributes, facility_attributes.	PostgreSQL with PostGIS + MongoDB
MULTIMEDIA	media_assets. Linking tables belong to their respective contexts below.	PostgreSQL for metadata + Amazon S3 for files
TRANSPORTATION_SENSORS	monitoring_units, sensors, traffic_readings, transit_events, capture_events, capture_media	PostgreSQL, with JSONB for device configuration
ENVIRONMENTAL_MONITORING	Collections: monitoring_devices, environmental_readings, environmental_events	MongoDB
CITIZEN_SERVICE_REQUESTS	service_requests, request_status_history, departments, request_media	PostgreSQL

Context / Schema	Main Proposed Tables or Collections	Proposed Technology
FACILITY_MAINTENANCE	inspections, work_orders, work_order_status_history, teams, work_order_costs, equipment_usage, inspection_media	PostgreSQL
EMERGENCY_RULES_AND_RESPONSE	response_rules, rule_versions, alerts, alert_actions, escalation_history	PostgreSQL + Rule Evaluation Service
HISTORICAL_PLANNING	Proposed analytical tables: fact_traffic_summary, fact_environmental_summary, fact_service_performance, fact_maintenance_cost, dim_date, dim_city, dim_facility	Separate PostgreSQL Analytical Database

A.1 Road And Facility Locations

Figure A1



Its responsibility is to maintain the platform's shared geographic foundation, allowing other contexts to identify cities, roads, public facilities, and service zones.

Arrows represent data associations rather than processing order. PostgreSQL maintains core records and geographic data; MongoDB stores supplementary attributes linked through shared identifiers.

Workflow example: Municipal staff register a park in facilities, linking it to a record in cities. Its geographic location is stored in PostGIS, while supplementary characteristics are stored in facility_attributes using the same identifier. Citizen requests and maintenance records can subsequently reference that park. Each core record belongs to one city in this proposed model. Roads crossing municipal boundaries are divided into segments. Service zones may overlap because different services can use different coverage boundaries. The application coordinates references and updates between PostgreSQL and MongoDB.

Table A3

Table / Collection	Information Stored	Business Justification
cities	City identifier, name, and geographic boundary.	Organize records by municipality and support comparisons across cities.
roads	Road segment identifier, city, name, category, operational status, and geometry.	Locate transportation readings, road-related requests, and interventions.
facilities	Facility identifier, city, name, category, operational status, and location.	Provide a common reference for service requests, inspections, and maintenance. Facilities include buildings, parks, parking lots, and bus stops.
service_zones	Zone identifier, city, service category, and geographic boundary.	Identify which service areas cover a given location.
road_attributes	Supplementary attributes that vary by road category.	Accommodate specific road characteristics without changing the core relational structure.
facility_attributes	Supplementary attributes that vary by facility category.	Accommodate details such as building capacity, park equipment, or bus stop amenities.

Cities

Each record represents a participating city.

Table A4

Field	Data Type	Required / Constraint	Purpose
city_id	UUID	Required; Primary Key	Uniquely identify the city.
name	TEXT	Required	Official city name.
boundary	GEOMETRY(MultiPolygon, 4326)	Optional	Represent the municipal boundary, when available.
created_at	TIMESTAMPTZ	Required	Record when the city was registered.
updated_at	TIMESTAMPTZ	Required	Record the latest update time.

Roads

Each record represents a road segment belonging to one city. Roads crossing municipal boundaries are divided into segments.

Table A5

Field	Data Type	Required / Constraint	Purpose
road_id	UUID	Required; Primary Key	Uniquely identify the road segment.
city_id	UUID	Required; Foreign Key → cities.city_id	Identify the city responsible for the segment.
name	TEXT	Optional	Road name, where available.

Field	Data Type	Required / Constraint	Purpose
road_type	TEXT	Required; controlled values	Classify the segment, for example local, arterial, or highway.
status	TEXT	Required; controlled values	Indicate its operational state, for example open, restricted, or closed.
geometry	GEOMETRY(LineString, 4326)	Required	Represent the geographic path of the segment.
created_at	TIMESTAMPTZ	Required	Record registration time.
updated_at	TIMESTAMPTZ	Required	Record the latest update time.

Facilities

Each record represents a public facility or infrastructure asset.

Table A6

Field	Data Type	Required / Constraint	Purpose
facility_id	UUID	Required; Primary Key	Uniquely identify the facility.
city_id	UUID	Required; Foreign Key → cities.city_id	Identify the city to which the facility belongs.
name	TEXT	Required	Facility name or descriptive label.
facility_type	TEXT	Required; controlled values	Identify the category: public_building, park, parking_lot, or bus_stop, for example.
status	TEXT	Required; controlled values	Indicate whether the facility is operational, restricted, or closed.
location	GEOMETRY(Point, 4326)	Required	Store a representative location, such as the main entrance.

Field	Data Type	Required / Constraint	Purpose
footprint	GEOMETRY(MultiPolygon, 4326)	Optional	Represent the full geographic area of a building, park, or parking lot.
created_at	TIMESTAMPTZ	Required	Record registration time.
updated_at	TIMESTAMPTZ	Required	Record the latest update time.

Service Zones

Each record represents a geographic coverage area for a service within one city.

Table A7

Field	Data Type	Required / Constraint	Purpose
zone_id	UUID	Required; Primary Key	Uniquely identify the zone.
city_id	UUID	Required; Foreign Key → cities.city_id	Identify the city to which the zone belongs.
name	TEXT	Required	Human-readable zone name.
service_type	TEXT	Required; controlled values	Identify the service, such as emergency_response, maintenance, or waste_collection.
boundary	GEOMETRY(MultiPolygon, 4326)	Required	Define the coverage area.
is_active	BOOLEAN	Required	Indicate whether the zone is currently in use.
created_at	TIMESTAMPTZ	Required	Record registration time.
updated_at	TIMESTAMPTZ	Required	Record the latest update time.

Road Attributes

Each optional document contains supplementary attributes for one road segment.

Table A8

Field	Data Type	Required / Constraint	Purpose
_id	String containing a UUID	Required; unique	Match the PostgreSQL road_id.
attributes	Object	Required	Contain the supplementary road characteristics.
attributes.surface_material	String	Optional	Describe the surface, such as asphalt or concrete.
attributes.lane_count	Integer	Optional; greater than zero	Record the number of lanes.
attributes.cycle_lane_present	Boolean	Optional	Indicate whether a cycle lane is present.
attributes.sidewalk_present	Boolean	Optional	Indicate whether a sidewalk is present.
created_at	BSON Date	Required	Record document creation time.
updated_at	BSON Date	Required	Record the latest update time.

Facility Attributes

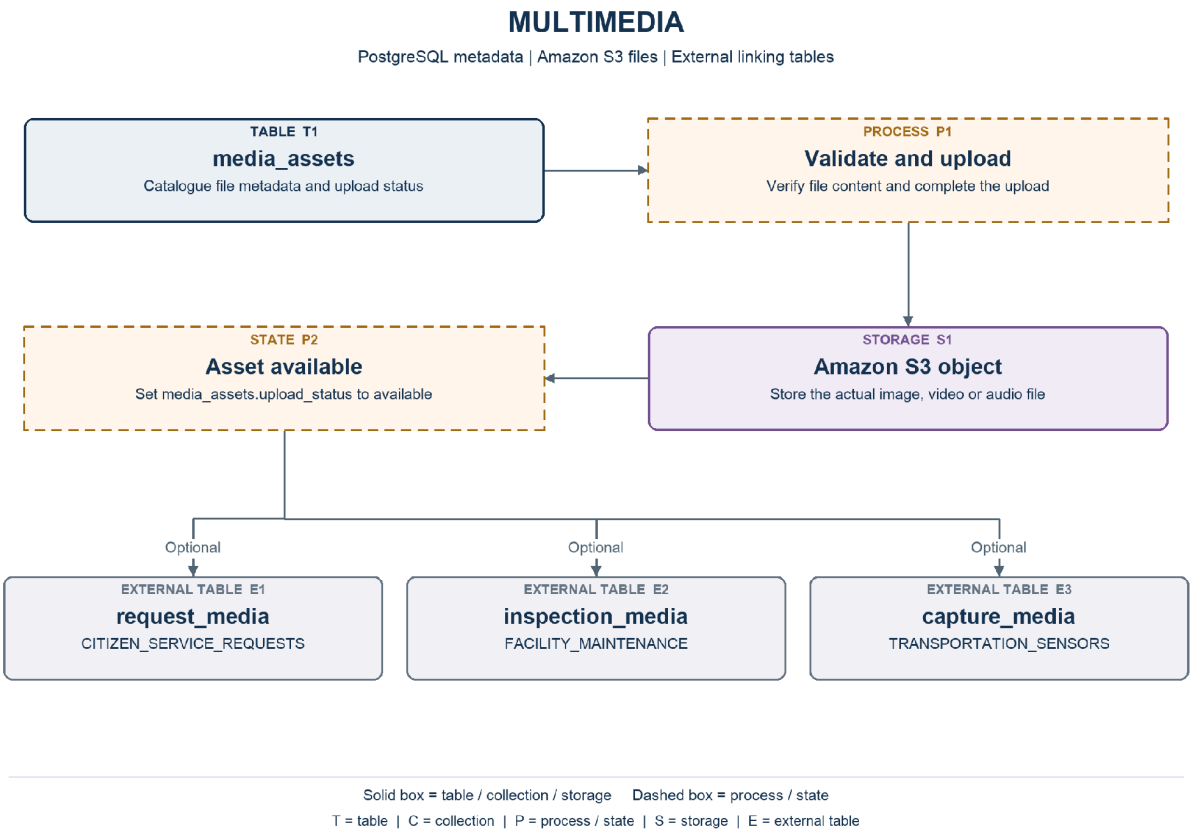
Each optional document contains supplementary attributes for one facility. Fields depend on its category.

Table A9

Field	Data Type	Required / Constraint	Purpose
_id	String containing a UUID	Required; unique	Match the PostgreSQL facility_id.
attributes	Object	Required	Contain category-specific facility characteristics.
attributes.building_capacity_persons	Integer	Optional; nonnegative	Record the capacity of a public building.
attributes.accessible_entrance	Boolean	Optional	Indicate whether an accessible entrance is available.
attributes.parking_space_count	Integer	Optional; nonnegative	Record the capacity of a parking facility.
attributes.park_amenities	Array of String	Optional	List amenities such as playground, picnic_area, or sports_court.
attributes.bus_stop_shelter	Boolean	Optional	Indicate whether a bus stop has a shelter.
created_at	BSON Date	Required	Record document creation time.
updated_at	BSON Date	Required	Record the latest update time.

A.2 Multimedia

Figure A2



Note. Solid boxes identify tables, collections, or storage. Dashed boxes identify processes or states. External components are labeled. Optional links are marked explicitly.

Responsibility: Receive and catalogue images and videos from citizens, inspectors, and cameras. Audio is an optional extension. The catalogue can exist before an asset is linked to a business record.

Workflow: Upload request → validate format and size → allocate an asset identifier → store the file in S3 → verify completion → mark the asset available → return its identifier to the originating service.

Tables and Collections

Table A10

Table / Collection	Information Stored	Business Justification
media_assets	Identify each file, its origin, storage location, and availability status.	Make evidence discoverable and traceable across channels without putting large files in operational tables.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Media Assets

Identify each file, its origin, storage location, and availability status.

Table A11

Field	Data Type	Required / Constraint	Purpose
media_id	UUID	Required; PK	Uniquely identify the asset.
media_type	TEXT	Required; controlled values	image or video; audio only if the extension is enabled.
mime_type	TEXT	Required	Validated content type, such as image/jpeg.
original_filename	TEXT	Required	Preserve the submitted filename for display.
size_bytes	BIGINT	Required; >= 0	Record the file size in bytes.

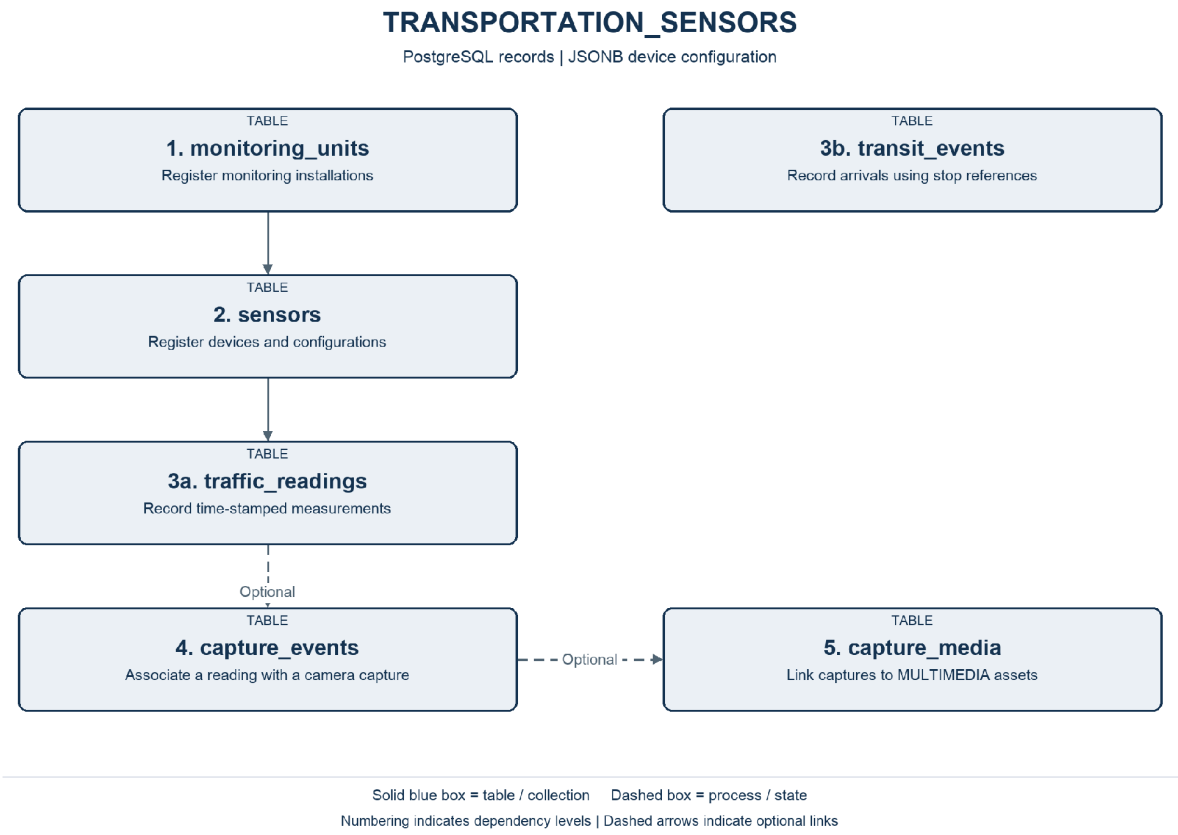
Field	Data Type	Required / Constraint	Purpose
source_channel	TEXT	Required; controlled values	citizen_portal, field_inspection, or traffic_camera.
storage_bucket	TEXT	Required	Name the S3 bucket.
storage_key	TEXT	Required; unique with bucket	Identify the object within the bucket.
upload_status	TEXT	Required; controlled values	pending, available, or failed.
checksum_sha256	TEXT	Optional	Record a verified checksum for integrity checks.
duration_seconds	NUMERIC(12, 3)	Optional; >= 0	Duration of a video or audio asset; absent for images.
captured_at	TIMESTAMPTZ	Optional	Capture time supplied by the source, when known.
received_at	TIMESTAMPTZ	Required	Time the upload was received.

Relationships and Validation

The application coordinates S3 and PostgreSQL because a database transaction cannot atomically commit both. Assets remain pending until the upload is verified; failed uploads are retried or cleaned up. Linking tables belong to their originating contexts. Multiple contexts may reference one asset. Files are private and retrieved through authorized application access.

A.3 Transportation Sensors

Figure A3



Note. Numbering indicates dependency levels. Solid blue boxes identify tables or collections; dashed boxes identify processes. Dashed arrows indicate optional links.

Responsibility: Maintain monitoring installations and devices, record individual or aggregate traffic measurements, and store transit arrival events. Camera capture is an optional extension to traffic monitoring.

Workflow: Register an installation and its devices → receive and validate readings or transit events → store measurements → present monitoring information → route relevant events to emergency rule evaluation. Optional camera captures link a reading to evidence.

Tables and Collections

Table A12

Table / Collection	Information Stored	Business Justification
monitoring_units	Group devices installed at the same monitoring point.	Keep device installations tied to a shared physical monitoring point.
sensors	Register radars, cameras, and other transportation devices.	Support equipment configuration, availability tracking, and maintenance.
traffic_readings	Store one measurement from one device at a specified time.	Enable comparisons of traffic conditions by device, place, and time.
transit_events	Record actual arrivals and compare them with schedules.	Measure punctuality and identify recurring transit delays.
capture_events	Associate an optional camera capture with a traffic reading.	Keep optional camera evidence associated with the relevant measurement for review.
capture_media	Associate capture events with multimedia assets.	Retrieve supporting files without duplicating their catalogue records.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Monitoring Units

Group devices installed at the same monitoring point.

Table A13

Field	Data Type	Required / Constraint	Purpose
unit_id	UUID	Required; PK	Identify the installation.
city_id	UUID	Required; external reference	Reference the responsible city.
road_id	UUID	Optional; external reference	Reference a road segment.
description	TEXT	Required	Describe the monitoring point.
lane_number	INTEGER	Optional; > 0	Identify the monitored lane, when applicable.
location	GEOMETRY(Point,4326)	Required	Position of the installation; requires PostGIS.
status	TEXT	Required; controlled values	active, inactive, or maintenance.
created_at	TIMESTAMPTZ	Required	Record creation time in UTC.
updated_at	TIMESTAMPTZ	Required	Latest update time in UTC.

Sensors

Register radars, cameras, and other transportation devices.

Table A14

Field	Data Type	Required / Constraint	Purpose
sensor_id	UUID	Required; PK	Identify the device.
unit_id	UUID	Required; local FK	Reference monitoring_units.unit_id.
sensor_type	TEXT	Required; controlled values	radar, camera, occupancy_detector, or other approved type.
status	TEXT	Required; controlled values	active, inactive, or maintenance.
configuration	JSONB	Required; JSON object	Store device-specific settings validated by sensor type.
configuration.sampling_interval_seconds	JSON integer	Radar; > 0 when present	Configure the interval between readings.
configuration.measured_variables	JSON array of strings	Radar; optional	List measurements, such as speed_kmh.
configuration.capture_mode	JSON string	Camera; optional	triggered or continuous.
configuration.resolution	JSON object	Camera; optional	Group output image dimensions.
configuration.resolution.width_px	JSON integer	Camera; > 0 when present	Image width in pixels.
configuration.resolution.height_px	JSON integer	Camera; > 0 when present	Image height in pixels.
created_at	TIMESTAMPTZ	Required	Record creation time in UTC.
updated_at	TIMESTAMPTZ	Required	Latest update time in UTC.

Traffic Readings

Store one measurement from one device at a specified time.

Table A15

Field	Data Type	Required / Constraint	Purpose
reading_id	UUID	Required; PK	Identify the reading.
sensor_id	UUID	Required; local FK	Reference the device producing the measurement.
source_event_id	TEXT	Required; unique per sensor	Prevent duplicate ingestion of the same source reading.
recorded_at	TIMESTAMPTZ	Required	Time of measurement.
measurement_type	TEXT	Required; controlled values	speed, occupancy, or congestion_level.
numeric_value	DOUBLE PRECISION	Conditional	Value for speed or occupancy; null for categorical readings.
category_value	TEXT	Conditional	Congestion category; null for numeric readings.
unit	TEXT	Conditional	km/h or percent for numeric readings.
lane_number	INTEGER	Optional; > 0	Lane measured.
vehicle_track_id	TEXT	Optional	Temporary equipment tracking identifier for individual vehicles.

Transit Events

Record actual arrivals and compare them with schedules.

Table A16

Field	Data Type	Required / Constraint	Purpose
transit_event_id	UUID	Required; PK	Identify the event.
stop_id	UUID	Required; external reference	Reference a bus stop in facilities.
route_id	TEXT	Required	Transit provider route identifier.
vehicle_id	TEXT	Required	Transit provider vehicle identifier.
source_event_id	TEXT	Required; unique with provider	Identify the source event for duplicate prevention.
provider_id	TEXT	Required	Identify the transit data provider.
scheduled_at	TIMESTAMPTZ	Optional	Expected arrival time, if supplied.
occurred_at	TIMESTAMPTZ	Required	Actual event time.

Capture Events

Associate an optional camera capture with a traffic reading.

Table A17

Field	Data Type	Required / Constraint	Purpose
capture_id	UUID	Required; PK	Identify the capture.
reading_id	UUID	Required; local FK	Reference the associated traffic reading.
camera_id	UUID	Required; local FK	Reference a sensor classified as a camera.
captured_at	TIMESTAMPTZ	Required	Image capture time.
vehicle_track_id	TEXT	Optional	Correlate the captured vehicle with the reading.
plate_text	TEXT	Optional	Automatically recognized plate text, if this extension is enabled.
plate_confidence	NUMERIC(5,4)	Optional; between 0 and 1	Confidence of the plate recognition result.
review_status	TEXT	Required; controlled values	pending, accepted, or rejected.

Capture Media

Associate capture events with multimedia assets.

Table A18

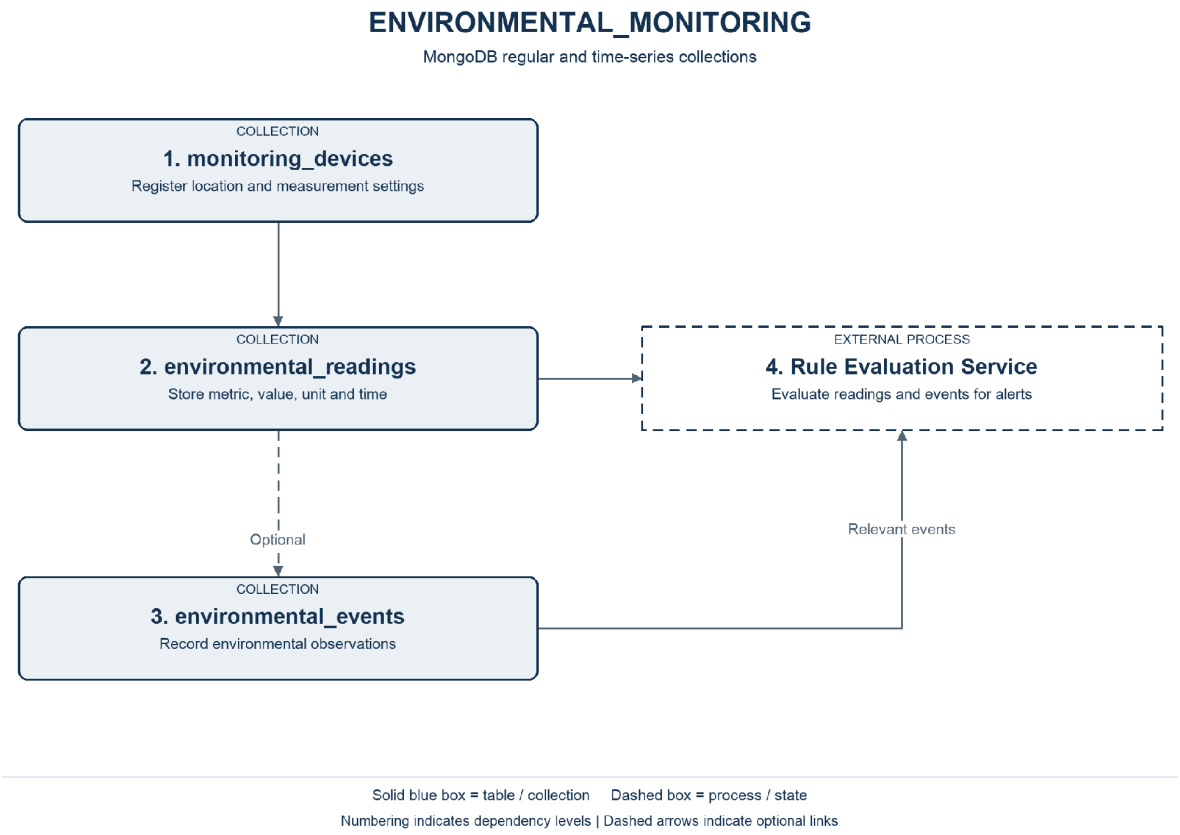
Field	Data Type	Required / Constraint	Purpose
capture_id	UUID	Required; composite PK; local FK	Reference the owning record.
media_id	UUID	Required; composite PK; external reference	Identify an asset in MULTIMEDIA.media_assets.
evidence_role	TEXT	Required	Describe the evidence role, such as before, after, or supporting.

Relationships and Validation

A sensor belongs to one monitoring unit. A unit references a city and optionally a road segment in the location context. JSONB configuration differs by device type. For radar it may contain sampling_interval_seconds (integer) and measured_variables (array of strings); for cameras, capture_mode (string) and resolution (object with integer width_px and height_px). Individual speed and aggregate occupancy are distinct measurement types. Camera-to-vehicle correlation requires validated timing, lane, and tracking information; an automatic plate result is not a confirmed violation. Local FKs enforce relationships only within this PostgreSQL context.

A.4 Environmental Monitoring

Figure A4



Note. Numbering indicates dependency levels. Solid blue boxes identify tables or collections; dashed boxes identify processes. Dashed arrows indicate optional links.

Responsibility: Maintain device configurations and record environmental measurements and observed events, supporting monitoring and emergency rule evaluation.

Workflow: Register a device → receive a measurement → validate device, metric, unit, and timestamp → store the reading → update monitoring views → forward relevant observations for emergency rule evaluation.

Tables and Collections

Table A19

Table / Collection	Information Stored	Business Justification
monitoring_devices	Register devices and their measurement capabilities.	Manage heterogeneous environmental devices and their capabilities.
environmental_readings	Store one environmental metric measured at one time.	Support trends and risk detection using traceable measurements.
environmental_events	Record environmental observations and source events.	Give operators contextual observations for investigation and response.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Monitoring Devices

Register devices and their measurement capabilities.

Table A20

Field	Data Type	Required / Constraint	Purpose
_id	String	Required; unique	Application-assigned device identifier.
city_id	String (UUID)	Required; external reference	Reference the city.
facility_id	String (UUID)	Optional; external reference	Reference the installation hosting the device.
location	Object	Required	GeoJSON point representing device position.
location.type	String	Required; value Point	Identify the GeoJSON geometry type.
location.coordinates	Array<Double>	Required; two values	Longitude followed by latitude.
status	String	Required; controlled values	active, inactive, or maintenance.
configuration	Object	Required	Device settings.
configuration.sampling_interval_seconds	Integer	Required; > 0	Expected measurement interval.
configuration.metrics	Array<Object>	Required	Supported measurements, each with metric (string) and unit (string).
updated_at	BSON Date	Required	Latest device update time.

Environmental Readings

Store one environmental metric measured at one time.

Table A21

Field	Data Type	Required / Constraint	Purpose
reading_id	String	Required; application identifier	Identify the reading for traceability.
recorded_at	BSON Date	Required; time field	Time the measurement was taken.
metadata	Object	Required; metadata field	Identify the measurement series.
metadata.device_id	String	Required; logical reference	Reference monitoring_devices._id.
metadata.city_id	String (UUID)	Required; external reference	Identify the responsible city.
metadata.metric	String	Required; controlled values	temperature, water_level, noise_level, or a specific air pollutant.
metadata.unit	String	Required	Unit appropriate to the metric, such as degC or dB.
value	Double	Required	Measured numeric value.
quality_status	String	Required; controlled values	valid, suspect, or invalid.
received_at	BSON Date	Required	Time the platform received the reading.

Environmental Events

Record environmental observations and source events.

Table A22

Field	Data Type	Required / Constraint	Purpose
_id	String	Required; unique	Identify the event.
device_id	String	Optional; logical reference	Reference the reporting device when applicable.
reading_id	String	Optional; logical reference	Reference a supporting measurement.
city_id	String (UUID)	Required; external reference	Identify the affected city.
event_type	String	Required	Classify the event, such as weather_event or elevated_water_level.
occurred_at	BSON Date	Required	Time of the observed event.
details	Object	Required	Store event-specific attributes; validate by event type.

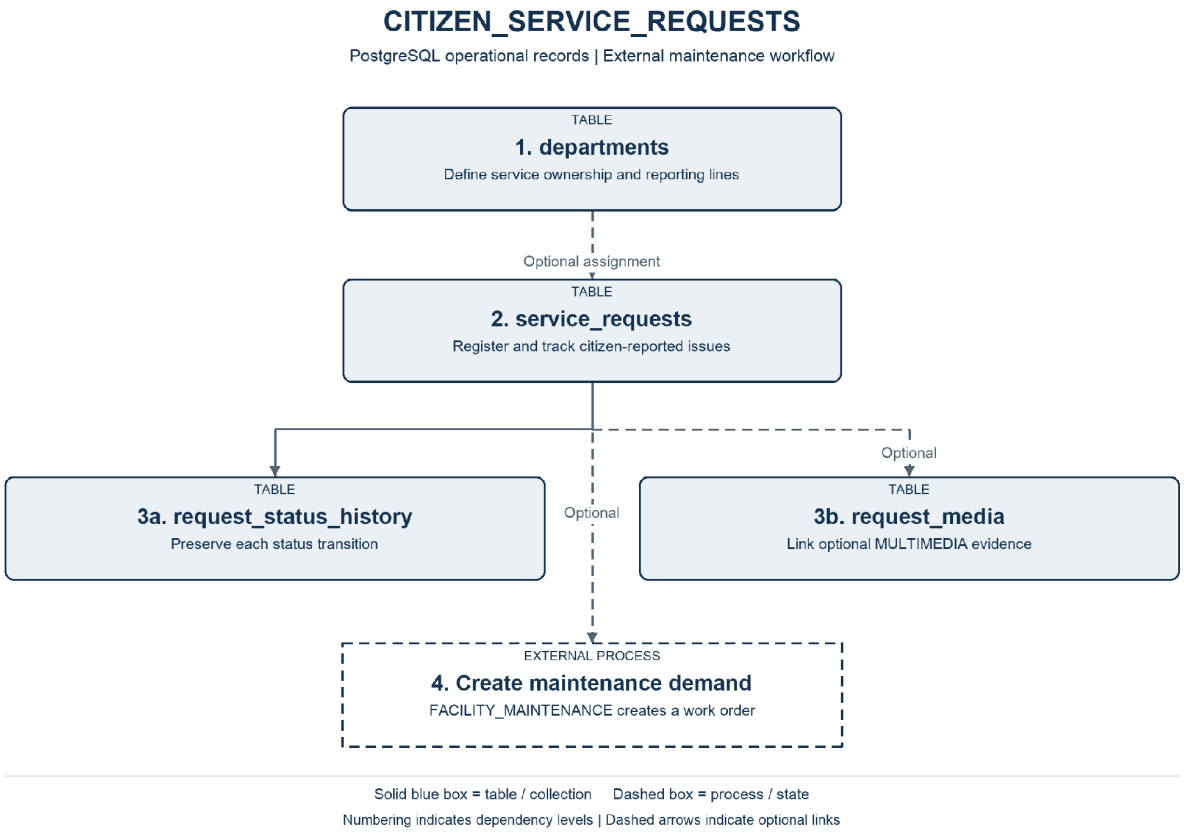
Relationships and Validation

Monitoring_devices and environmental_events are regular collections. environmental_readings is a proposed time-series collection using recorded_at as its time field and metadata as its metadata field. The application generates reading_id for traceability; ingestion duplicate handling is an application responsibility. UUIDs from PostgreSQL are represented consistently as strings. Environmental events represent observations; response alerts

are owned by EMERGENCY_RULES_AND_RESPONSE. Missing measurements are not interpreted as zero.

A.5 Citizen Service Requests

Figure A5



Note. Solid boxes identify tables, collections, or storage. Dashed boxes identify processes or states. External components are labelled. Optional links are marked explicitly.

Responsibility: Register citizen-reported issues, assign departmental ownership, preserve status changes, and link optional supporting evidence.

Workflow: Citizen submits issue details and location → system validates and issues a tracking number → staff triage and assign a department → staff update progress → maintenance is requested where necessary → resolution is recorded and the request is closed.

Tables and Collections

Table A23

Table / Collection	Information Stored	Business Justification
departments	Identify departments responsible for service delivery.	Establish clear ownership and support departmental reporting structures.
service_requests	Store each citizen-reported issue and its current state.	Provide reliable citizen issue tracking and accountable assignment.
request_status_history	Preserve the progression of a request.	Explain progress, delays, and resolution through an auditable history.
request_media	Link requests to optional supporting files.	Make citizen evidence available to the team handling the request.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Departments

Identify departments responsible for service delivery.

Table A24

Field	Data Type	Required / Constraint	Purpose
department_id	UUID	Required; PK	Identify the department.
city_id	UUID	Required; external reference	Reference the responsible city.
parent_department_id	UUID	Optional; self FK	Represent a parent department where a hierarchy exists.
name	TEXT	Required	Department name.
is_active	BOOLEAN	Required	Whether the department accepts assignments.

Service Requests

Store each citizen-reported issue and its current state.

Table A25

Field	Data Type	Required / Constraint	Purpose
request_id	UUID	Required; PK	Identify the request.
tracking_number	TEXT	Required; unique	Provide a public tracking reference.
submitted_by_ref	TEXT	Optional	Opaque identity reference; anonymous reports may omit it.
city_id	UUID	Required; external reference	Reference the city.
road_id	UUID	Optional; external reference	Reference an affected road segment.
facility_id	UUID	Optional; external reference	Reference an affected facility.
department_id	UUID	Optional; local FK	Reference the assigned department after triage.
category	TEXT	Required; controlled values	Classify the issue, such as pothole or broken_streetlight.
description	TEXT	Required	Describe the reported problem.
location	GEOMETRY(Point,4326)	Required	Position of the issue; requires PostGIS.
priority	TEXT	Required; controlled values	low, normal, high, or urgent.
status	TEXT	Required; controlled values	submitted, triaged, assigned, in_progress, resolved, or closed.
resolved_at	TIMESTAMPTZ	Optional	Time resolution was recorded.

Field	Data Type	Required / Constraint	Purpose
created_at	TIMESTAMPTZ	Required	Record creation time in UTC.
updated_at	TIMESTAMPTZ	Required	Latest update time in UTC.

Request Status History

Preserve the progression of a request.

Table A26

Field	Data Type	Required / Constraint	Purpose
history_id	UUID	Required; PK	Identify the transition.
request_id	UUID	Required; local FK	Reference service_requests.request_id.
from_status	TEXT	Optional	Previous status; absent for initial registration.
to_status	TEXT	Required	New status.
changed_by_ref	TEXT	Required	Identity or service performing the transition.
changed_at	TIMESTAMPTZ	Required	Time of the transition.
note	TEXT	Optional	Explain the action or resolution.

Request Media

Link requests to optional supporting files.

Table A27

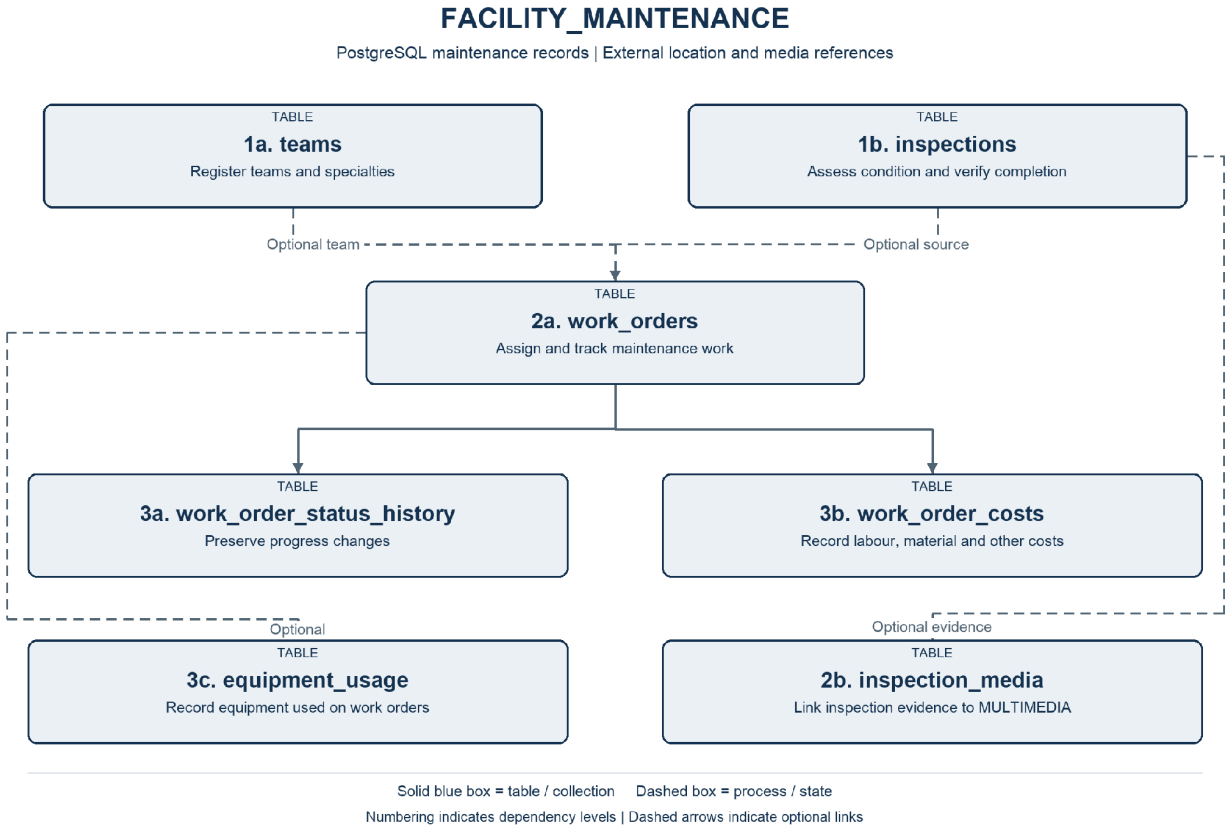
Field	Data Type	Required / Constraint	Purpose
request_id	UUID	Required; composite PK; local FK	Reference the owning record.
media_id	UUID	Required; composite PK; external reference	Identify an asset in MULTIMEDIA.media_assets.
evidence_role	TEXT	Required	Describe the evidence role, such as before, after, or supporting.

Relationships and Validation

City, road, and facility identifiers reference the location context. A request may identify a precise location without referring to an existing facility. Submitter references are opaque identifiers managed by an identity service; personal contact information is outside this dictionary. Current status and its history are updated in one local transaction. A request can lead to several work orders, each referencing the originating request. Status transitions and priority values are validated by application rules.

A.6 Facility Maintenance

Figure A6



Note. Numbering indicates dependency levels. Solid blue boxes identify tables or collections; dashed boxes identify processes. Dashed arrows indicate optional links.

Responsibility: Coordinate inspections and preventive or corrective maintenance, tracking team assignments, work progress, costs, equipment, and inspection evidence.

Workflow: Inspection, citizen request, or preventive schedule creates a demand → coordinator opens a work order → assigns a team and priority → team records work, equipment use, and costs → supervisor verifies completion → work order closes and the originating request is notified where applicable.

Tables and Collections

Table A28

Table / Collection	Information Stored	Business Justification
teams	Identify maintenance teams and their specialties.	Support assignment of work to an appropriate maintenance specialty.
inspections	Record an inspection of a public facility.	Document facility condition and support verification of completed repairs.
work_orders	Plan and track maintenance execution.	Coordinate execution, priority, scheduling, and completion of maintenance.
work_order_status_history	Preserve work order status transitions.	Preserve the progression and responsibility for maintenance actions.
work_order_costs	Record individual cost items for a work order.	Support expenditure oversight and comparable cost reporting.
equipment_usage	Record equipment used during maintenance.	Provide accountability for equipment deployed during maintenance.
inspection_media	Link inspection findings to multimedia evidence.	Support assessment and verification with photographic or video evidence.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Teams

Identify maintenance teams and their specialties.

Table A29

Field	Data Type	Required / Constraint	Purpose
team_id	UUID	Required; PK	Identify the team.
department_id	UUID	Required; external reference	Reference the responsible department.
name	TEXT	Required	Team name.
specialty	TEXT	Required	Primary maintenance specialty.
is_active	BOOLEAN	Required	Whether the team is available for assignment.

Inspections

Record an inspection of a public facility.

Table A30

Field	Data Type	Required / Constraint	Purpose
inspection_id	UUID	Required; PK	Identify the inspection.
facility_id	UUID	Required; external reference	Reference the inspected facility.
inspector_ref	TEXT	Required	Identity reference of the inspector.
inspected_at	TIMESTAMPTZ	Required	Time of inspection.
inspection_type	TEXT	Required; controlled values	routine, issue_assessment, or completion_verification.

Field	Data Type	Required / Constraint	Purpose
outcome	TEXT	Required; controlled values	satisfactory, action_required, or follow_up.
findings	TEXT	Required	Describe observations and required action.

Work Orders

Plan and track maintenance execution.

Table A31

Field	Data Type	Required / Constraint	Purpose
work_order_id	UUID	Required; PK	Identify the work order.
facility_id	UUID	Required; external reference	Reference the maintained facility.
source_request_id	UUID	Optional; external reference	Reference an originating citizen request.
source_inspection_id	UUID	Optional; local FK	Reference an inspection that identified the issue.
completion_inspection_id	UUID	Optional; local FK	Reference a completion verification inspection.
team_id	UUID	Optional; local FK	Reference the assigned team.
description	TEXT	Required	Describe the work to perform.
maintenance_type	TEXT	Required; controlled values	preventive or corrective.
priority	TEXT	Required; controlled values	low, normal, high, or urgent.

Field	Data Type	Required / Constraint	Purpose
status	TEXT	Required; controlled values	open, assigned, in_progress, awaiting_verification, completed, or cancelled.
scheduled_for	TIMESTAMPTZ	Optional	Planned start time.
completed_at	TIMESTAMPTZ	Optional	Verified completion time.
created_at	TIMESTAMPTZ	Required	Record creation time in UTC.
updated_at	TIMESTAMPTZ	Required	Latest update time in UTC.

Work Order Status History

Preserve work order status transitions.

Table A32

Field	Data Type	Required / Constraint	Purpose
history_id	UUID	Required; PK	Identify the transition.
work_order_id	UUID	Required; local FK	Reference the work order.
from_status	TEXT	Optional	Previous status.
to_status	TEXT	Required	New status.

Field	Data Type	Required / Constraint	Purpose
changed_by_ref	TEXT	Required	Identity or service performing the change.
changed_at	TIMESTAMPTZ	Required	Transition time.
note	TEXT	Optional	Explain the action.

Work Order Costs

Record individual cost items for a work order.

Table A33

Field	Data Type	Required / Constraint	Purpose
cost_id	UUID	Required; PK	Identify the cost item.
work_order_id	UUID	Required; local FK	Reference the work order.
cost_category	TEXT	Required; controlled values	labour, materials, equipment, or external_service.
description	TEXT	Required	Describe the cost item.
amount	NUMERIC(14, 2)	Required; >= 0	Actual cost amount.
currency_code	CHAR(3)	Required	Currency of the cost, such as CAD.
recorded_at	TIMESTAMPTZ	Required	Cost recording time.

Equipment Usage

Record equipment used during maintenance.

Table A34

Field	Data Type	Required / Constraint	Purpose
usage_id	UUID	Required; PK	Identify the usage record.
work_order_id	UUID	Required; local FK	Reference the work order.
equipment_ref	TEXT	Required	Reference equipment in the municipal asset register.
started_at	TIMESTAMPTZ	Required	Start of equipment use.
ended_at	TIMESTAMPTZ	Optional; >= started_at	End of use, when known.
note	TEXT	Optional	Describe how the equipment was used.

Inspection Media

Link inspection findings to multimedia evidence.

Table A35

Field	Data Type	Required / Constraint	Purpose
inspection_id	UUID	Required; composite PK; local FK	Reference the owning record.
media_id	UUID	Required; composite PK; external reference	Identify an asset in MULTIMEDIA.media_assets.

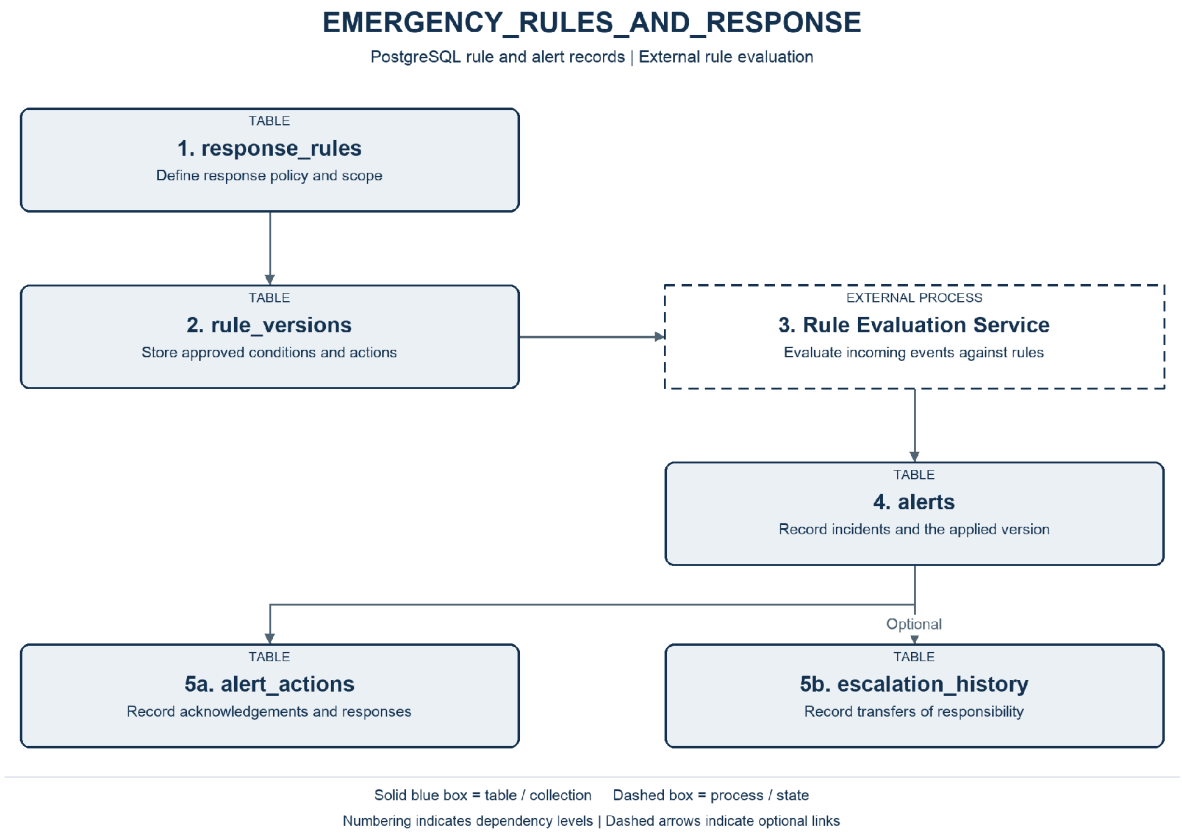
Field	Data Type	Required / Constraint	Purpose
evidence_role	TEXT	Required	Describe the evidence role, such as before, after, or supporting.

Relationships and Validation

Facility references are owned by ROAD_AND_FACILITY_LOCATIONS and department references by CITIZEN_SERVICE_REQUESTS. inspections may create work orders; work orders may instead originate from a request or preventive activity. No source inspection or request is mandatory for preventive work. inspection_media links before or after inspection evidence. Completion is verified by an inspection linked through completion_inspection_id. Teams and work orders are locally related. Monetary values are stored with a currency code and are not aggregated across currencies without an explicit conversion policy.

A.7 Emergency Rules and Response

Figure A7



Note. Numbering indicates dependency levels. Solid blue boxes identify tables or collections; dashed boxes identify processes. Dashed arrows indicate optional links.

Responsibility: Maintain approved response rules, evaluate incoming events, and preserve alerts, response actions, and escalation history.

Workflow: Create a rule → approve an immutable version → receive an event → evaluate conditions against the applicable version → record an alert → acknowledge and act → escalate when required → close the alert.

Tables and Collections

Table A36

Table / Collection	Information Stored	Business Justification
response_rules	Identify response policies and their scope.	Provide governed, clearly scoped response policies.
rule_versions	Preserve approved conditions and response settings.	Make decisions reproducible using the approved conditions applied at the time.
alerts	Record each condition requiring a response.	Keep incidents requiring action visible and prevent duplicate response records.
alert_actions	Track acknowledgements and response activities.	Record acknowledgement and follow-through by responsible personnel.
escalation_history	Track transfers to additional response authorities.	Show why and when responsibility was escalated.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Response Rules

Identify response policies and their scope.

Table A37

Field	Data Type	Required / Constraint	Purpose
rule_id	UUID	Required; PK	Identify the policy.

Field	Data Type	Required / Constraint	Purpose
name	TEXT	Required	Rule name.
description	TEXT	Required	Business purpose of the rule.
source_context	TEXT	Required; controlled values	Context whose events the rule evaluates.
city_id	UUID	Optional; external reference	Limit the rule to a city; absent for regional scope.
zone_id	UUID	Optional; external reference	Limit the rule to a service zone.
is_active	BOOLEAN	Required	Whether the policy is enabled.
created_at	TIMESTAMPTZ	Required	Record creation time in UTC.
updated_at	TIMESTAMPTZ	Required	Latest update time in UTC.

Rule Versions

Preserve approved conditions and response settings.

Table A38

Field	Data Type	Required / Constraint	Purpose
rule_version_id	UUID	Required; PK	Identify the version.
rule_id	UUID	Required; local FK	Reference response_rules.rule_id.
version_number	INTEGER	Required; unique per rule	Sequence of versions.
conditions	JSONB	Required; validated object	metric (string), operator (string), threshold (number), unit (string), and optional window_seconds (integer).
priority	TEXT	Required; controlled values	Priority assigned when the condition is met.
target_department_ref	TEXT	Required	Identify the initial responding department.
escalation_after_seconds	INTEGER	Optional; > 0	Delay before escalation if unresolved.
approval_status	TEXT	Required; controlled values	draft, approved, or retired.
approved_by_ref	TEXT	Conditional	Approver identity; required for approved versions.
effective_from	TIMESTAMPTZ	Required	Start of version applicability.
effective_to	TIMESTAMPTZ	Optional; > effective_from	End of version applicability.

Alerts

Record each condition requiring a response.

Table A39

Field	Data Type	Required / Constraint	Purpose
alert_id	UUID	Required; PK	Identify the alert.
rule_version_id	UUID	Required; local FK	Reference the exact rule version evaluated.
source_context	TEXT	Required	Identify the originating context.
source_event_id	TEXT	Required	Identifier of the event or reading evaluated.
deduplication_key	TEXT	Required; unique	Prevent duplicate alerts for the same evaluation.
priority	TEXT	Required	Priority captured at alert creation.
status	TEXT	Required; controlled values	open, acknowledged, in_progress, or closed.
triggered_at	TIMESTAMPTZ	Required	Time the alert was generated.
closed_at	TIMESTAMPTZ	Optional	Time response was closed.

Alert_actions

Track acknowledgements and response activities.

Table A40

Field	Data Type	Required / Constraint	Purpose
action_id	UUID	Required; PK	Identify the action.
alert_id	UUID	Required; local FK	Reference alerts.alert_id.
action_type	TEXT	Required	acknowledged, team_dispatched, status_changed, or closed.
performed_by_ref	TEXT	Required	Identity or service performing the action.
performed_at	TIMESTAMPTZ	Required	Time of action.
note	TEXT	Optional	Describe the action or outcome.

Escalation History

Track transfers to additional response authorities.

Table A41

Field	Data Type	Required / Constraint	Purpose
escalation_id	UUID	Required; PK	Identify the escalation.
alert_id	UUID	Required; local FK	Reference the alert.
from_department_ref	TEXT	Required	Previous responsible department.
to_department_ref	TEXT	Required	Department receiving the escalation.
reason	TEXT	Required	Explain why escalation occurred.

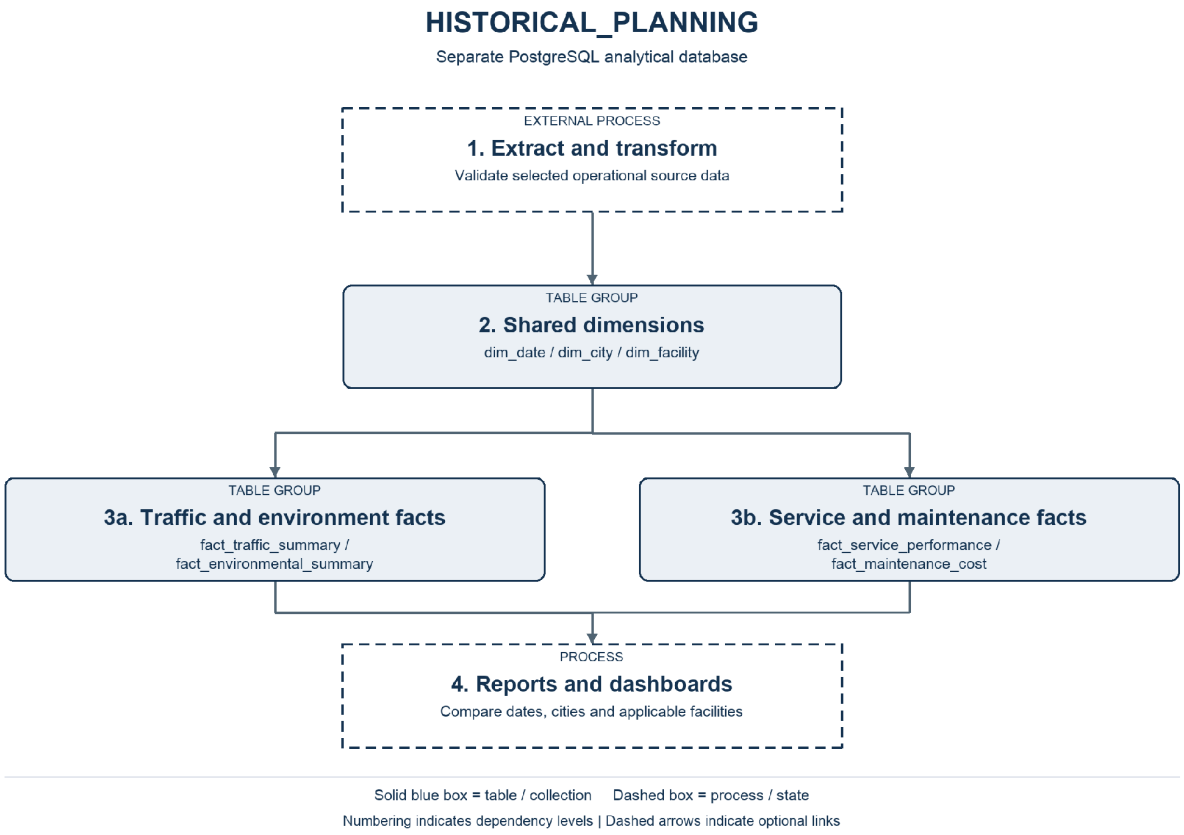
Field	Data Type	Required / Constraint	Purpose
escalated_at	TIMESTAMPTZ	Required	Time of escalation.

Relationships and Validation

The response_rules identifies a policy; rule_versions stores immutable versions. conditions is a JSONB object validated against an approved rule language, not executable SQL supplied by users. The external service performs evaluation; this design does not claim that PostgreSQL triggers execute the rule engine. Each alert records the exact version applied and the source context plus event identifier. Recording an alert and its initial action should be coordinated locally; notification delivery uses retry and deduplication. Only approved, effective versions can be evaluated.

A.8 Historical Planning

Figure A8



Note. Numbering indicates dependency levels. Solid blue boxes identify tables or collections; dashed boxes identify processes. Dashed arrows indicate optional links.

Responsibility: Consolidate operational information into daily indicators for regional planning without placing reporting workloads on operational stores.

Workflow: Extract changes from operational sources → validate identifiers, dates, units, and completeness → load dimensions → aggregate and upsert daily facts → publish reports with a visible refresh time.

Tables and Collections

Table A42

Table / Collection	Information Stored	Business Justification
dim_date	Provide a shared reporting calendar.	Make reporting periods consistent across operational domains.
dim_city	Provide a common city dimension.	Enable consistent comparisons between participating cities.
dim_facility	Provide facility context for maintenance indicators.	Connect facility characteristics to maintenance expenditure.
fact_traffic_summary	Grain: one city, road segment, and reporting date.	Support road-level trend comparisons for transportation planning.
fact_environmental_summary	Grain: one city, metric, unit, and reporting date.	Support environmental comparisons using explicit metrics and units.
fact_service_performance	Grain: one city, request category, and reporting date.	Assess service demand and resolution performance by city and category.
fact_maintenance_cost	Grain: one facility, currency, and cost-recording date.	Support maintenance budgets and facility investment decisions.

Fields and Constraints

PK = primary key; FK = foreign key. An external reference is validated by the application or integration service, rather than a foreign key across independent databases. Field definitions below are the proposed logical design.

Dim Date

Provide a shared reporting calendar.

Table A43

Field	Data Type	Required / Constraint	Purpose
date_key	INTEGER	Required; PK	Calendar key in YYYYMMDD form.
calendar_date	DATE	Required; unique	Actual reporting date.
year_number	SMALLINT	Required	Calendar year.
month_number	SMALLINT	Required; 1 to 12	Calendar month.
day_of_week	SMALLINT	Required; 1 to 7	ISO weekday number.

Dim City

Provide a common city dimension.

Table A44

Field	Data Type	Required / Constraint	Purpose
city_key	BIGINT	Required; surrogate PK	Identify the analytical city record.
source_city_id	UUID	Required; unique	Original city identifier.
city_name	TEXT	Required	City name used for reporting.

Dim Facility

Provide facility context for maintenance indicators.

Table A45

Field	Data Type	Required / Constraint	Purpose
facility_key	BIGINT	Required; surrogate PK	Identify the analytical facility record.
source_facility_id	UUID	Required; unique	Original facility identifier.
city_key	BIGINT	Required; local FK	Reference dim_city.city_key.
facility_name	TEXT	Required	Facility display name.
facility_type	TEXT	Required	Facility category.

Fact Traffic Summary

Grain: one city, road segment, and reporting date.

Table A46

Field	Data Type	Required / Constraint	Purpose
traffic_summary_id	BIGINT	Required; surrogate PK	Identify the daily summary.
date_key	INTEGER	Required; local FK	Reference dim_date.date_key.
city_key	BIGINT	Required; local FK	Reference dim_city.city_key.
source_road_id	UUID	Required; grain key	Original road segment identifier.
speed_sample_count	BIGINT	Required; >= 0	Number of valid speed samples.
average_speed_kmh	DOUBLE PRECISION	Optional	Average of comparable valid speed readings.

Field	Data Type	Required / Constraint	Purpose
average_occupancy_percentage	DOUBLE PRECISION	Optional; 0 to 100	Average valid occupancy over the reporting period.
loaded_at	TIMESTAMPTZ	Required	Latest successful load time.

Fact Environmental Summary

Grain: one city, metric, unit, and reporting date.

Table A47

Field	Data Type	Required / Constraint	Purpose
environmental_summary_id	BIGINT	Required; surrogate PK	Identify the daily summary.
date_key	INTEGER	Required; local FK	Reference dim_date.date_key.
city_key	BIGINT	Required; local FK	Reference dim_city.city_key.
metric	TEXT	Required; grain key	Specific environmental variable.
unit	TEXT	Required; grain key	Measurement unit; incompatible units are not combined.
sample_count	BIGINT	Required; >= 0	Number of valid samples.
minimum_value	DOUBLE PRECISION	Optional	Minimum valid observed value.
maximum_value	DOUBLE PRECISION	Optional	Maximum valid observed value.
average_value	DOUBLE PRECISION	Optional	Average of comparable samples; not an area-weighted city estimate.
loaded_at	TIMESTAMPTZ	Required	Latest successful load time.

Fact Service Performance

Grain: one city, request category, and reporting date.

Table A48

Field	Data Type	Required / Constraint	Purpose
service_summary_id	BIGINT	Required; surrogate PK	Identify the daily summary.
date_key	INTEGER	Required; local FK	Reference dim_date.date_key.
city_key	BIGINT	Required; local FK	Reference dim_city.city_key.
request_category	TEXT	Required; grain key	Service request category.
submitted_count	BIGINT	Required; >= 0	Requests submitted on this date.
resolved_count	BIGINT	Required; >= 0	Requests resolved on this date.
total_resolution_seconds	NUMERIC(20, 0)	Required; >= 0	Sum of creation-to-resolution durations for requests resolved on this date.
loaded_at	TIMESTAMPTZ	Required	Latest successful load time.

Fact Maintenance Cost

Grain: one facility, currency, and cost-recording date.

Table A49

Field	Data Type	Required / Constraint	Purpose
maintenance_summary_id	BIGINT	Required; surrogate PK	Identify the daily summary.
date_key	INTEGER	Required; local FK	Reference dim_date.date_key.
facility_key	BIGINT	Required; local FK	Reference dim_facility.facility_key.
currency_code	CHAR(3)	Required; grain key	Currency used for the cost total.
cost_item_count	BIGINT	Required; >= 0	Number of cost entries recorded on this date.
total_cost	NUMERIC(18, 2)	Required; >= 0	Sum of cost entries in the specified currency.
loaded_at	TIMESTAMPTZ	Required	Latest successful load time.

Relationships and Validation

Daily fact tables use a unique constraint on their declared grain. Integration maps source identifiers to analytical keys and loads dimensions before facts. There are no cross-database foreign keys. Group dates in one documented reporting timezone and display the latest successful refresh time. Preserve missing measurements as unknown. Facts join to their applicable dimensions, not to other fact tables.

Appendix B

Entity Relationship Diagrams

These diagrams expand the operational and analytical relationships discussed in Part 2. Figures B1–B8 correspond to the eight data sources, including SQL foreign keys and application-managed references.

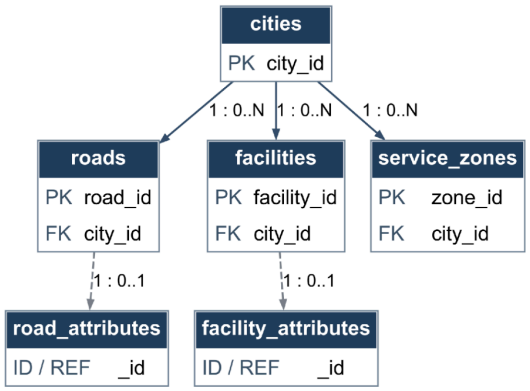
Entity Relationships

B.1 ROAD_AND_FACILITY_LOCATIONS

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier

Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B1



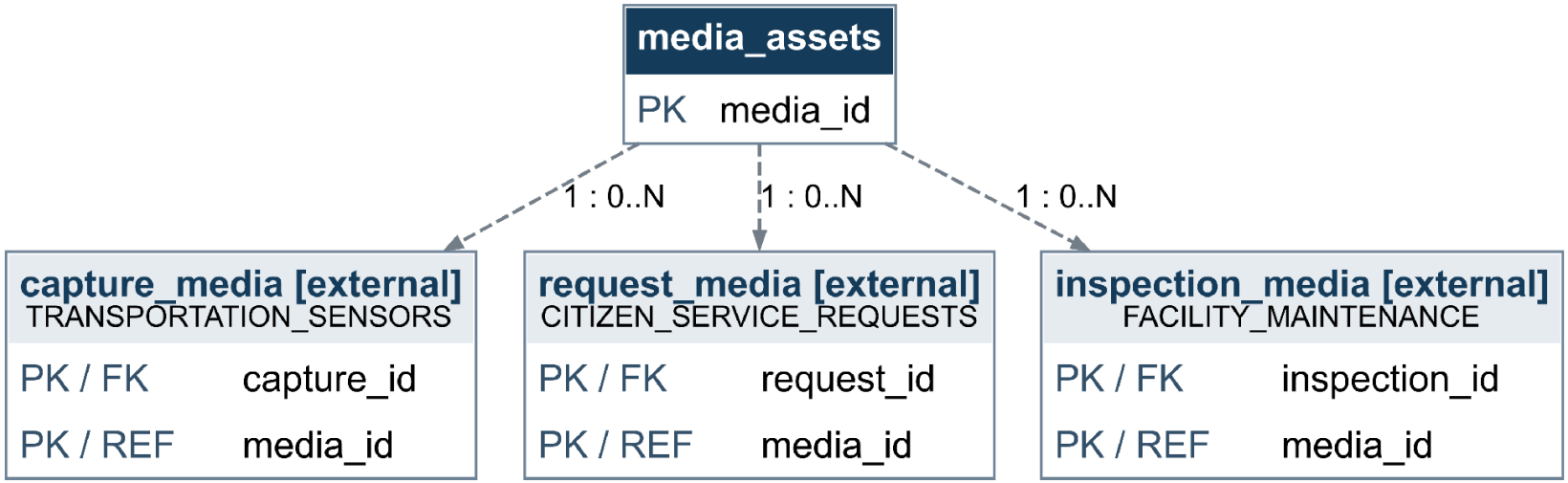
Shared identifiers link each optional MongoDB attribute document to exactly one road or facility. Each parent has at most one such document.

B.2 MULTIMEDIA

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier

Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B2

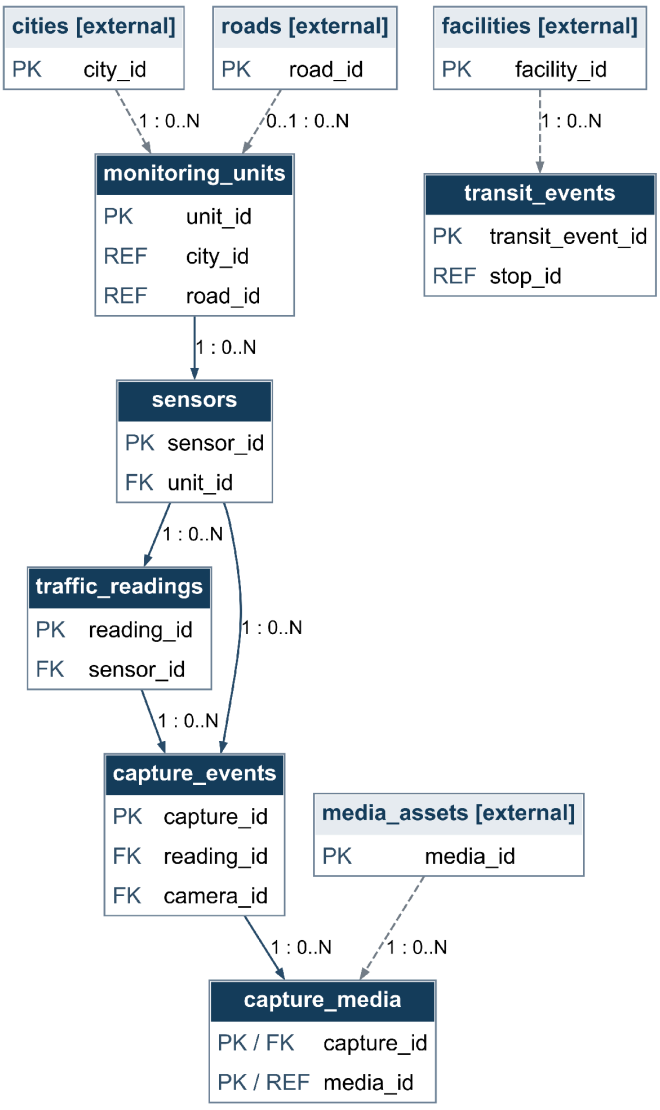


MULTIMEDIA contains only media_assets. capture_media belongs to TRANSPORTATION_SENSORS; request_media belongs to CITIZEN_SERVICE_REQUESTS; inspection_media belongs to FACILITY_MAINTENANCE. These external linking tables reference media_assets. S3 objects are files, not relational tables.

B.3 TRANSPORTATION_SENSORS

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B3

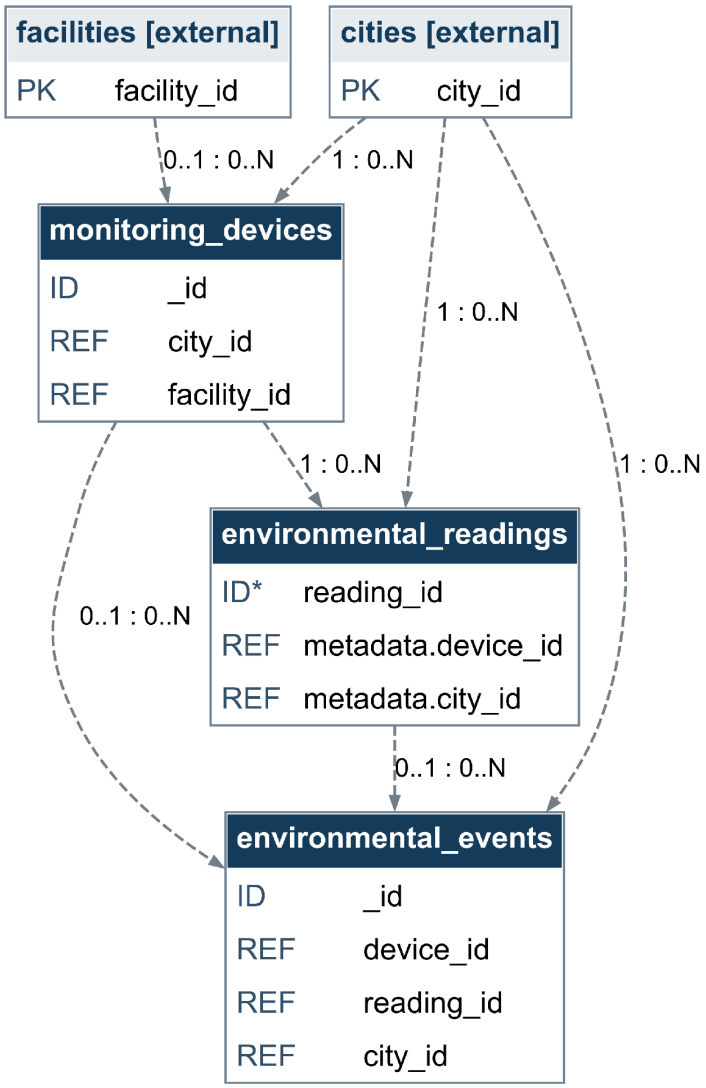


camera_id references a sensor registered as a camera. A reading may have several capture events; reading_id is not unique in capture_events

B.4 ENVIRONMENTAL_MONITORING

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier. Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B4



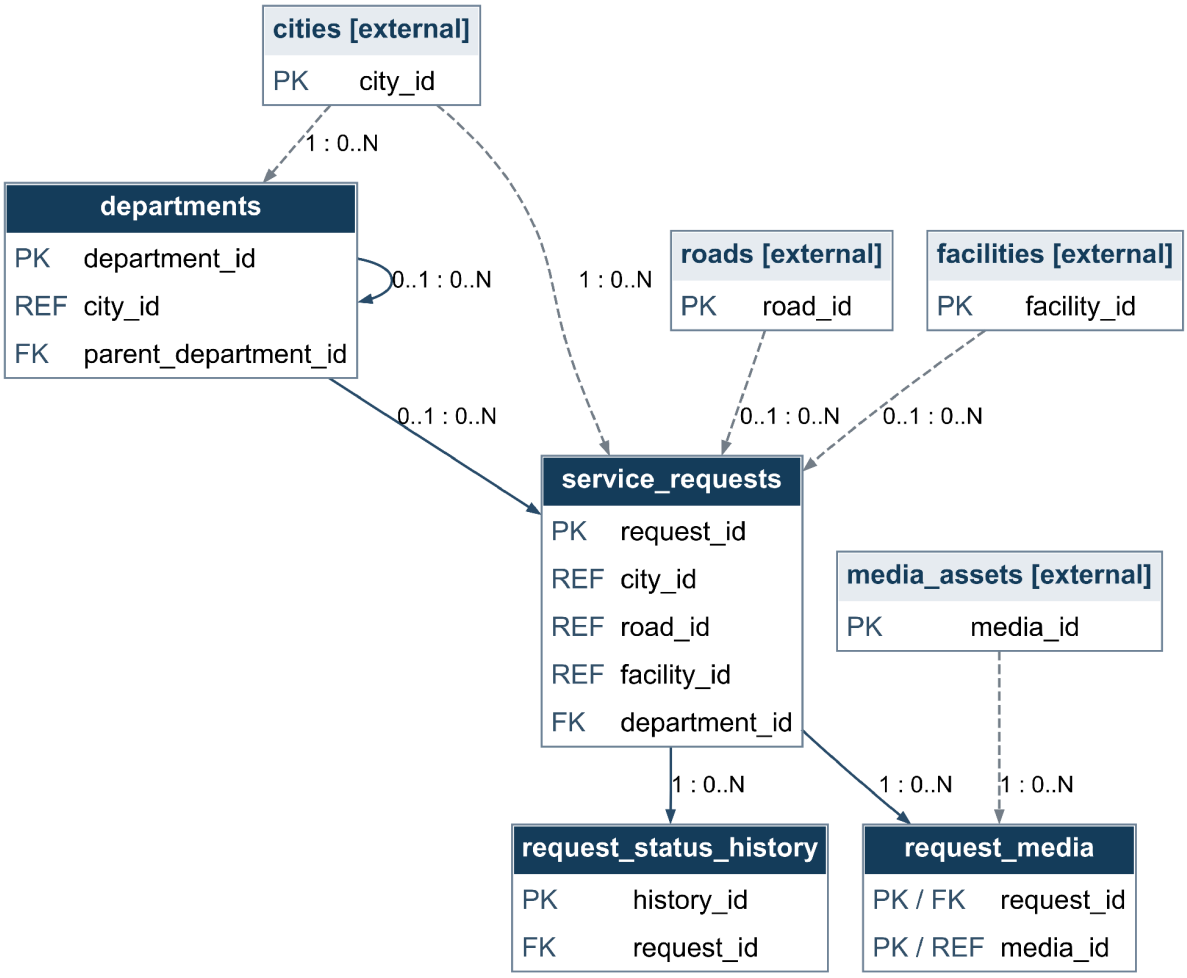
MongoDB collections use logical references, not SQL FKs. ID* reading_id is an application traceability identifier; uniqueness is not enforced by this time-series design.

B.5 CITIZEN_SERVICE_REQUESTS

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier

Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B5



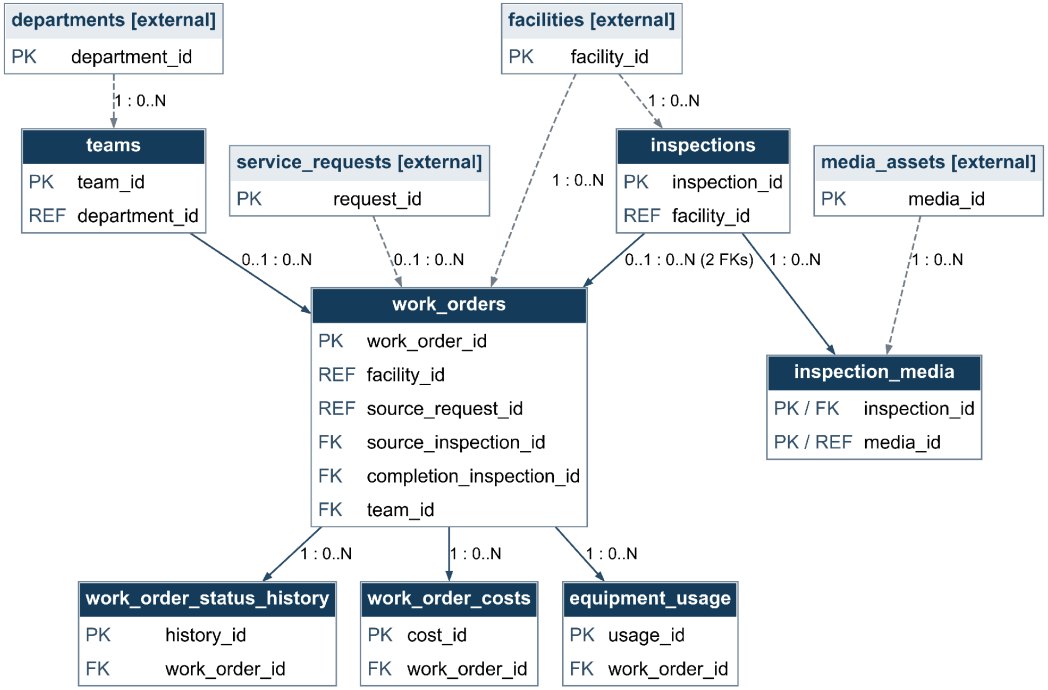
A request may be unassigned and may reference a road or facility. The department hierarchy allows one optional parent and many children; cycles must be prevented.

B.6 FACILITY_MAINTENANCE

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier

Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B6

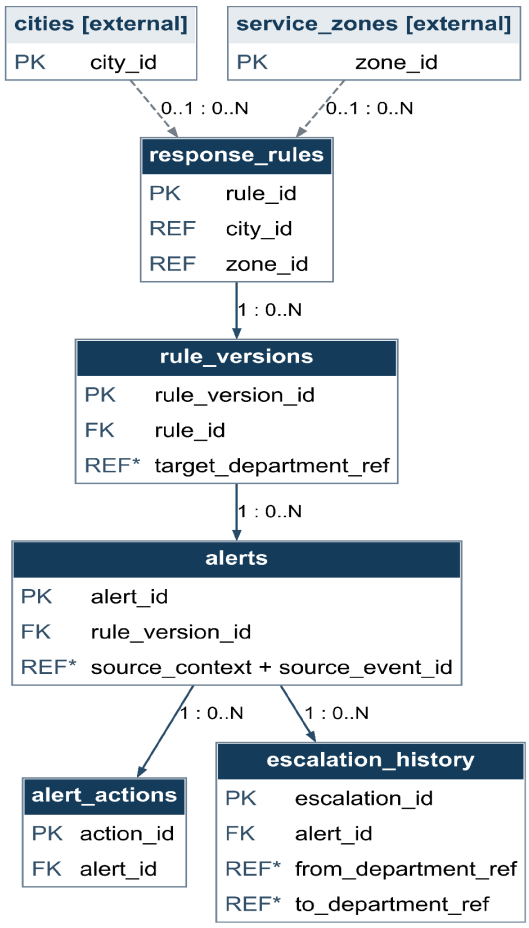


source_inspection_id and completion_inspection_id are separate optional relationships to inspections. Neither field is unique, so neither relationship is one-to-one.

B.7 EMERGENCY_RULES_AND_RESPONSE

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier. Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B7



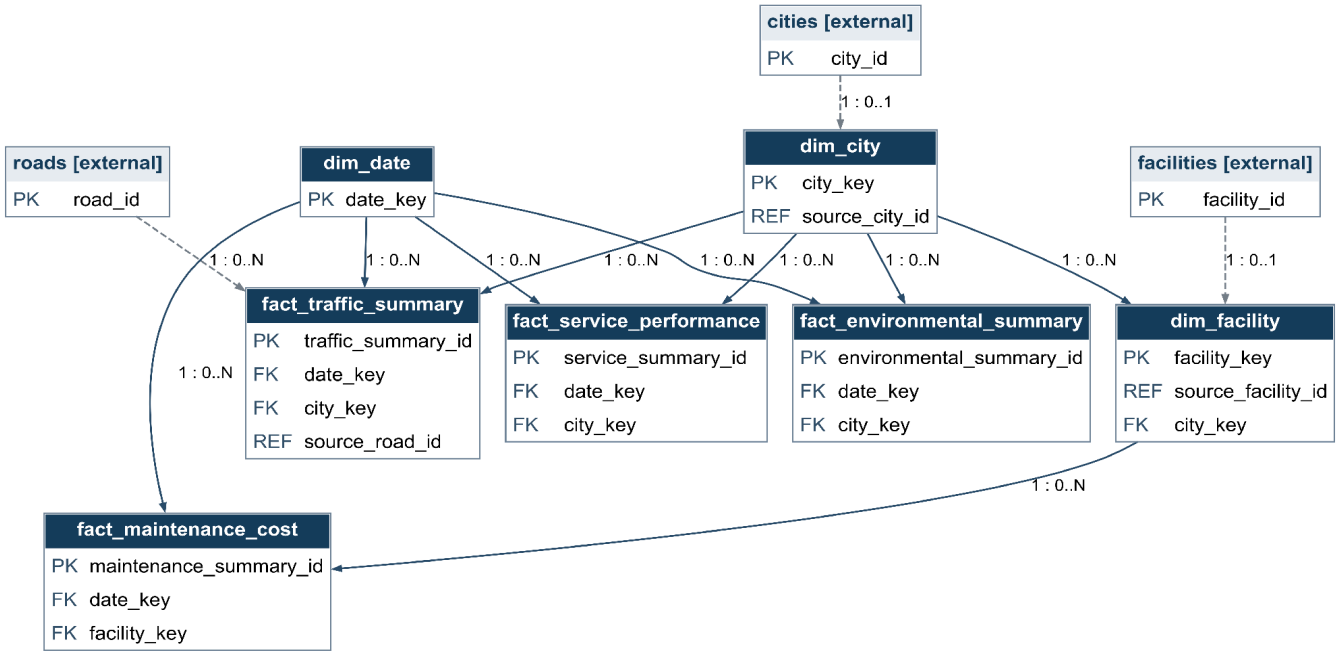
REF* identifies application-managed references. source_context with source_event_id identifies an event in the originating context; it is not a SQL FK. Department references are resolved by the application.

B.8 HISTORICAL_PLANNING

PK = primary key FK = foreign key (solid line) REF = logical reference (dashed) ID = document identifier

Arrow: parent to child. Label: parent count : child count. 1 = one; 0..1 = optional one; 0..N = zero or many.

Figure B8



Facts reference shared dimensions. Unique source_city_id and source_facility_id allow at most one current dimension record per source entity. Dashed links show lineage references, not database FKs.

Appendix C

Database Scripts and CRUD Examples

This appendix contains the implementation instructions and complete CRUD examples referenced in Part 3. Table C1 links to the eight native database scripts in the GitHub repository. Sections C.1–C.8 preserve the examples for each source.

Implementation and Execution Order

The operational schemas share srsp_operational; the historical schema uses srsp_analytics. MongoDB databases are srsp_location_attributes and srsp_environment. Script 02 stores the S3 bucket and object key as metadata; the upload service handles the files separately. All eight scripts are available in the project's GitHub repository under part3/scripts (main branch); the table links directly to each file (mehrthur-teste, n.d.).

Table C1

Order	Context	Executable script
01	Locations	01_road_and_facility_locations.sql
02	Multimedia	02_multimedia.sql
03	Transportation	03_transportation_sensors.sql
04	Environment	04_environmental_monitoring.js
05	Citizen requests	05_citizen_service_requests.sql
06	Maintenance	06_facility_maintenance.sql
07	Emergency response	07_emergency_rules_and_response.sql
08	Historical planning	08_historical_planning.sql

Run scripts 01–08 in order using psql and mongosh as described in part3/README.md. The scripts use concrete demonstration values; application integrations should bind submitted values as parameters. UUIDs identify operational records; MongoDB stores shared UUIDs as strings. Primary keys, unique constraints, checks and foreign keys protect the relational model. CREATE SCHEMA, CREATE

TABLE and CREATE INDEX use IF NOT EXISTS. Database creation checks the catalogue because PostgreSQL has no CREATE DATABASE IF NOT EXISTS syntax.

Seven native PostgreSQL SQL files and one native mongosh JavaScript file implement all eight sources. Each SQL table has explicit INSERT, SELECT, UPDATE and DELETE statements. The MongoDB file also implements optional location attributes. Disposable records demonstrate deletion while retaining shared parents. Existing schemas require explicit migrations if their structure differs; IF NOT EXISTS does not repair incompatible tables.

The following SQL and MongoDB examples illustrate one representative record per source. They assume the tables and parent records already created by the executable scripts. Named values such as :request_id are application parameters; substitute bound values through the relevant driver. Run the DELETE examples only on disposable demonstration records. Full scripts also cover the supporting tables and collection validation.

C.1 CRUD Example 1 Road and Facility Locations

Executable script: 01_road_and_facility_locations.sql

PostGIS stores core geography; optional MongoDB attribute documents use the same UUID and are validated against SQL parents.

```
-- CREATE a facility linked to an existing city
INSERT INTO road_and_facility_locations.facilities
 (facility_id, city_id, name, facility_type, status, location)
VALUES (:facility_id, :city_id, 'Demo park', 'park',
 'operational', ST_SetSRID(ST_MakePoint(-79.38,43.65),4326));
-- READ facilities within 1 kilometre
SELECT facility_id, name
FROM road_and_facility_locations.facilities
WHERE ST_DWithin(location::geography,
 ST_SetSRID(ST_MakePoint(-79.38,43.65),4326)::geography,1000);
-- UPDATE descriptive information
UPDATE road_and_facility_locations.facilities
SET name='Reviewed park', updated_at=now()
WHERE facility_id=:facility_id;
-- DELETE only an unreferenced demonstration facility
DELETE FROM road_and_facility_locations.facilities
WHERE facility_id=:facility_id;
```

C.2 CRUD Example 2 Multimedia

Executable script: 02_multimedia.sql

The SQL script implements catalogue CRUD. Seed metadata remains pending because no object is uploaded by SQL. The upload service performs S3 PutObject, GetObject and DeleteObject, then verifies the checksum before marking a file available.

```
-- CREATE metadata before uploading the S3 object
INSERT INTO multimedia.media_assets
 (media_id,media_type,mime_type,original_filename,size_bytes,
 source_channel,storage_bucket,storage_key,upload_status)
VALUES (:media_id,'image','image/png','evidence.png',:size,
 'citizen_portal',:bucket,:object_key,'pending');
-- READ the catalogue entry
SELECT media_id,storage_bucket,storage_key,upload_status
FROM multimedia.media_assets WHERE media_id=:media_id;
-- UPDATE only after successful upload and checksum verification
UPDATE multimedia.media_assets
SET upload_status='available',checksum_sha256=:checksum
WHERE media_id=:media_id;
-- Before file deletion, mark unavailable and unlink consumers.
-- Delete the S3 object, then DELETE its disposable metadata.
DELETE FROM multimedia.media_assets WHERE media_id=:media_id;
```

C.3 CRUD Example 3 Transportation Sensors

Executable script: 03_transportation_sensors.sql

A reading belongs to a sensor, which belongs to a monitoring unit and road. Source-event uniqueness prevents duplicate ingestion. The full script includes transit and camera evidence.

```
-- CREATE a reading for an existing sensor
INSERT INTO transportation_sensors.traffic_readings
 (reading_id,sensor_id,source_event_id,recorded_at,
 measurement_type,numeric_value,unit)
VALUES (:reading_id,:sensor_id,:source_event_id,:recorded_at,
 'speed',62.5,'km/h');
-- READ a bounded sensor history
SELECT recorded_at,numeric_value,unit
FROM transportation_sensors.traffic_readings
WHERE sensor_id=:sensor_id
 AND recorded_at>=:start_time AND recorded_at<:end_time
ORDER BY recorded_at DESC LIMIT 100;
-- UPDATE a corrected disposable reading
UPDATE transportation_sensors.traffic_readings
SET numeric_value=61.0 WHERE reading_id=:reading_id;
-- DELETE after removing any dependent capture links/events
DELETE FROM transportation_sensors.traffic_readings
WHERE reading_id=:reading_id;
```

C.4 CRUD Example 4 Environmental Monitoring

Executable script: 04_environmental_monitoring.js

The native mongosh script creates and validates regular collections and creates the time-series collection. It implements device/event CRUD and reading insertion, bounded retrieval, metadata update and temporary-series deletion. Optional location-attribute CRUD is included in the same file.

```
// CREATE a regular device document
const env = db.getSiblingDB('srsp_environment');
env.monitoring_devices.insertOne({
 _id: deviceId, city_id: cityId,
 location: {type:'Point',coordinates:[-79.3832,43.6532]},
 status:'active',
 configuration:{sampling_interval_seconds:10,
  metrics:[{metric:'temperature',unit:'degC'}]},
 updated_at:new Date()
});
// READ
 env.monitoring_devices.findOne({_id:deviceId});
// UPDATE
 env.monitoring_devices.updateOne({_id:deviceId},
  {$set:{status:'maintenance',updated_at:new Date()}});
// DELETE only after clearing reading/event references
 env.monitoring_devices.deleteOne({_id:deviceId});
```

C.5 CRUD Example 5 Citizen Service Requests

Executable script: 05_citizen_service_requests.sql

The full SQL script includes departments, requests, status history and media links. The demonstrated transition locks a disposable request and changes its expected previous state together with history in one transaction.

```
-- CREATE with existing city, department and optional facility
INSERT INTO citizen_service_requests.service_requests
 (request_id,tracking_number,city_id,department_id,
 category,description,location,priority,status)
VALUES (:request_id,:tracking_number,:city_id,:department_id,
 'broken_streetlight','Broken light',:point,'normal','submitted');
-- READ by public tracking reference
SELECT * FROM citizen_service_requests.service_requests
WHERE tracking_number=:tracking_number;
-- UPDATE state and history in one transaction after validation
BEGIN;
SELECT status FROM citizen_service_requests.service_requests
WHERE request_id=:request_id FOR UPDATE;
UPDATE citizen_service_requests.service_requests
SET status='triaged',updated_at=now() WHERE request_id=:request_id;
INSERT INTO citizen_service_requests.request_status_history
 (history_id,request_id,from_status,to_status,changed_by_ref)
```

```
VALUES (:history_id,:request_id,'submitted','triaged',:actor);
COMMIT;
-- DELETE a disposable child history before its parent
DELETE FROM citizen_service_requests.request_status_history
WHERE request_id=:request_id;
DELETE FROM citizen_service_requests.service_requests
WHERE request_id=:request_id;
```

C.6 CRUD Example 6 Facility Maintenance

Executable script: 06_facility_maintenance.sql

Work orders can reference a request and source inspection; preventive orders may omit both. Completion needs a satisfactory verification inspection for the same facility. Status transitions also write history atomically.

```
-- CREATE with existing facility, team and optional request
INSERT INTO facility_maintenance.work_orders
 (work_order_id,facility_id,source_request_id,team_id,
 description,maintenance_type,priority,status)
VALUES (:work_order_id,:facility_id,:request_id,:team_id,
 'Repair light','corrective','normal','open');
-- READ a facility history
SELECT work_order_id,status,created_at
FROM facility_maintenance.work_orders
WHERE facility_id=:facility_id ORDER BY created_at DESC;
-- UPDATE the descriptive scope
UPDATE facility_maintenance.work_orders
SET description='Replace damaged light',updated_at=now()
WHERE work_order_id=:work_order_id;
-- DELETE a disposable unreferenced work order
-- Remove its costs, equipment usage and status history first.
DELETE FROM facility_maintenance.work_orders
WHERE work_order_id=:work_order_id;
```

C.7 CRUD Example 7 Emergency Rules and Response

Executable script: 07_emergency_rules_and_response.sql

Approved versions are immutable. The CRUD update uses a draft version; alerts retain a reference to an approved effective version. The service validates source events and responding departments.

```
-- CREATE an editable draft for an existing rule
INSERT INTO emergency_rules_and_response.rule_versions
 (rule_version_id,rule_id,version_number,conditions,priority,
 target_department_ref,approval_status,effective_from)
VALUES (:version_id,:rule_id,:version_number,
 '{"metric":"speed","operator":">","threshold":60,"unit":"km/h"}',
 'high',:department_ref,'draft',:effective_from);
-- READ the rule version
SELECT * FROM emergency_rules_and_response.rule_versions
WHERE rule_version_id=:version_id;
```

```
-- UPDATE only a draft; published versions require a new version
UPDATE emergency_rules_and_response.rule_versions
SET conditions=jsonb_set(conditions,'{threshold}','65')
WHERE rule_version_id=:version_id AND approval_status='draft';
-- DELETE an unreferenced disposable draft
DELETE FROM emergency_rules_and_response.rule_versions
WHERE rule_version_id=:version_id AND approval_status='draft';
```

C.8 CRUD Example 8 Historical Planning

Executable script: 08_historical_planning.sql

Connect to srsp_analytics. Load dimensions before facts and map source UUIDs to local surrogate keys. No foreign key or SQL join crosses the operational and analytical databases.

```
-- CREATE a daily fact after loading its date/city dimensions
INSERT INTO historical_planning.fact_traffic_summary
 (date_key,city_key,source_road_id,speed_sample_count,
 average_speed_kmh,loaded_at)
VALUES (:date_key,:city_key,:road_id,100,45.5,now())
ON CONFLICT (date_key,city_key,source_road_id) DO NOTHING;
-- READ by reporting period and city
SELECT d.calendar_date,f.average_speed_kmh,f.loaded_at
FROM historical_planning.fact_traffic_summary f
JOIN historical_planning.dim_date d USING(date_key)
WHERE f.city_key=:city_key
 AND d.calendar_date BETWEEN :start_date AND :end_date;
-- UPDATE a recomputed daily value without duplicating its grain
UPDATE historical_planning.fact_traffic_summary
SET speed_sample_count=101,average_speed_kmh=45.7,loaded_at=now()
WHERE date_key=:date_key AND city_key=:city_key
 AND source_road_id=:road_id;
-- DELETE only the disposable demonstration fact
DELETE FROM historical_planning.fact_traffic_summary
WHERE date_key=:date_key AND city_key=:city_key
 AND source_road_id=:road_id;
```

Appendix D

Indexing Retrieval Queries

The eight complete retrieval queries supporting the Part 4 indexing table appear in Sections D.1–D.8. Each section retains its original explanation and limitations; query execution and measured performance are not claimed.

Example Retrieval Queries

The six SQL and two mongosh examples use replaceable sample inputs. SQL targets srsp_operational; MongoDB targets srsp_environment. The queries and proposed indexes require server-side validation.

D.1 Scenario 1: Open Requests in a City From Oldest to Newest

```
SELECT
    request_id,
    tracking_number,
    status,
    department_id,
    created_at
FROM citizen_service_requests.service_requests
WHERE city_id = UUID '00000000-0000-0000-0000-000000000001'
  AND status IN (
      'submitted',
      'triaged',
      'assigned',
      'in_progress'
  )
ORDER BY created_at ASC, request_id ASC;
```

Returns creation date/time; request_id breaks timestamp ties. The existing queue index filters but needs a global sort. The proposed active-request partial index (city_id, created_at, request_id) can support the requested order when justified by workload.

D.2 Scenario 2: Sensor Readings During a Period

```
SELECT
    reading_id,
    recorded_at,
    measurement_type,
    numeric_value,
    unit
FROM transportation_sensors.traffic_readings
WHERE sensor_id = UUID '00000000-0000-0000-0000-000000000015'
  AND recorded_at >= TIMESTAMPTZ '2026-10-23 00:00:00+00'
  AND recorded_at <  TIMESTAMPTZ '2026-10-24 00:00:00+00'
ORDER BY recorded_at DESC;
```

Matches traffic_sensor_time_idx(sensor_id, recorded_at DESC). The exclusive end avoids overlap between consecutive periods.

D.3 Scenario 3: Facilities in a Zone From Nearest to Farthest

```
SELECT
    f.facility_id,
    f.name,
    f.facility_type,
    ST_Distance(
        f.location::geography,
        'SRID=4326;POINT(-79.3832 43.6532)'::geography
    ) AS distance_metres
```

```
FROM road_and_facility_locations.service_zones AS z
JOIN road_and_facility_locations.facilities AS f
    ON f.city_id = z.city_id
   AND ST_Covers(z.boundary, f.location)
WHERE z.zone_id = UUID '00000000-0000-0000-0000-000000000005'
ORDER BY distance_metres ASC, f.facility_id ASC;
```

Replace the sample longitude/latitude with the selected or incident location. ST_Covers includes boundary points and the city condition preserves zone ownership. The proposed GiST(location) filters spatial candidates; ST_Distance on geography calculates spheroidal distance in metres, followed by an explicit sort. This is geographic distance, not driving distance. facility_id breaks distance ties consistently.

D.4 Scenario 4: Facility Work Orders Over Two Years

```
SELECT
    work_order_id,
    status,
    maintenance_type,
    created_at
FROM facility_maintenance.work_orders
WHERE facility_id = UUID '00000000-0000-0000-0000-000000000003'
  AND created_at >=
      TIMESTAMPTZ '2026-10-24 00:00:00+00' - INTERVAL '2 years'
  AND created_at <= TIMESTAMPTZ '2026-10-24 00:00:00+00'
ORDER BY created_at DESC;
```

Matches work_facility_time_idx(facility_id, created_at DESC). The fixed as-of instant makes the example repeatable; replace both occurrences together. All statuses are included.

D.5 Scenario 5: Available Evidence for a Request

```
SELECT
    m.media_id,
    rm.evidence_role,
    m.media_type,
    m.original_filename,
    m.storage_bucket,
    m.storage_key
FROM citizen_service_requests.request_media AS rm
JOIN multimedia.media_assets AS m
    ON m.media_id = rm.media_id
WHERE rm.request_id = UUID '00000000-0000-0000-0000-00000000001f'
  AND m.upload_status = 'available';
```

Reuses request_media(request_id, media_id) and media_assets(media_id) primary-key indexes. Availability is checked on matching metadata; the application authorises S3 access. Part 3 seed uploads remain pending, so an empty result is expected until verified uploads are available.

D.6 Scenario 6: Environmental Readings Above a Threshold

```
db.getSiblingDB("srsp_environment").environmental_readings.find(
  {
    "metadata.city_id": "00000000-0000-0000-0000-000000000001",
    "metadata.metric": "temperature",
    "metadata.unit": "degC",
    recorded_at: {
      $gte: ISODate("2026-10-23T00:00:00Z"),
      $lt: ISODate("2026-10-24T00:00:00Z")
```

```
    },
    value: { $gt: 20 },
    quality_status: "valid"
  },
  {
    _id: 0,
    reading_id: 1,
    recorded_at: 1,
    value: 1,
    metadata: 1
  }
);
```

Illustrates a temperature threshold of 20 degrees Celsius, not an emergency policy. The proposed city/metric/unit/time index narrows candidate buckets; value and quality remain residual filters.

D.7 Scenario 7: Unacknowledged Urgent Alerts

```
SELECT
    alert_id,
    source_context,
    source_event_id,
    triggered_at
FROM emergency_rules_and_response.alerts
WHERE status = 'open'
  AND priority = 'urgent'
ORDER BY triggered_at ASC
LIMIT 50;
```

The literal status/priority conditions match the proposed partial index predicate. It supports the oldest-first queue when the subset is selective. LIMIT 50 gives one result batch; repeated polling requires application claim/acknowledgement handling.

D.8 Scenario 8: Active Devices Near an Incident

```
db.getSiblingDB("srsp_environment").monitoring_devices.find(
  {
    status: "active",
    location: {
      $near: {
        $geometry: {
          type: "Point",
          coordinates: [-79.3832, 43.6532]
        },
        $maxDistance: 2000
      }
    }
  },
  {
    _id: 1,
    city_id: 1,
    location: 1,
    status: 1
  }
).limit(20);
```

Reuses device_location_geo on location (2dsphere). The example returns at most 20 active devices within 2,000 metres, nearest first; no additional sort is needed. Status is not part of this index.